

Computer Vision

Classification

Dr. Adduru U G Sankararao

**Indian Institute of Information Technology Sri City,
Chittoor**

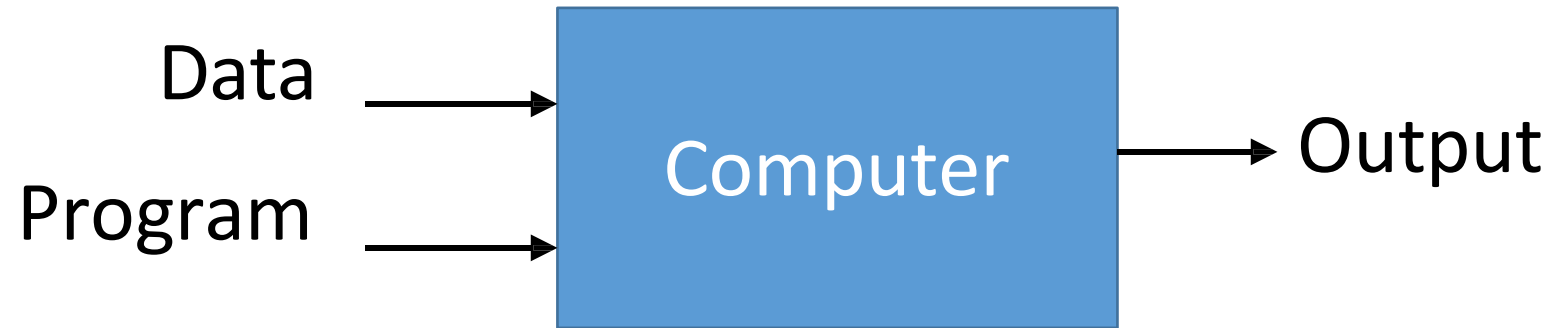


What is Machine Learning?

- Making predictions or decisions from data
- “Programming computers to optimize a performance criterion using example data or past experience” (Ethem Alpaydin, Machine Learning, 2010)
- “A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .” (Tom Mitchell, Machine Learning, 1997)
- “Learning general models from a data of particular examples”
- “Build a model that is *a good and useful approximation* to the data.”

ML

Traditional Programming



Machine Learning



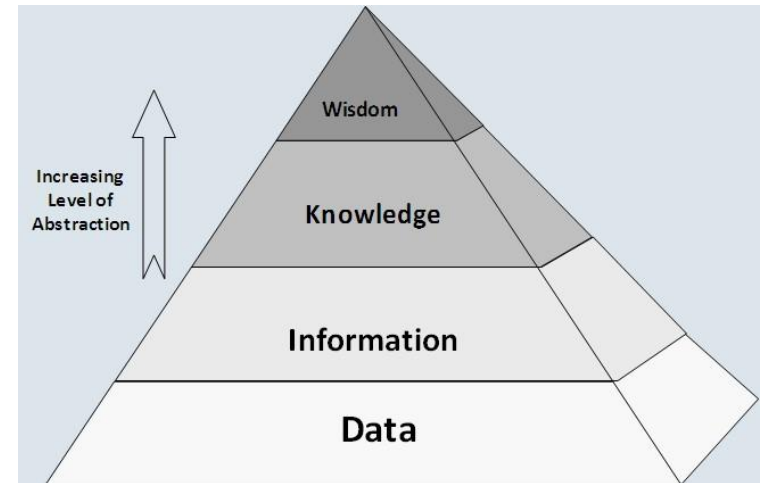
Related Terms

- Machine Learning,
- Data Mining,
- Knowledge Discovery,
- Artificial Intelligence,
- Statistical Learning,
- Pattern Recognition,
- Computational Learning



When is Machine Learning Used?

- Human expertise does not exist
 - E.g. navigating on Mars
- Humans are unable to explain their expertise
 - E.g. speech recognition
- Solution changes in time
 - E.g. routing on a computer network
- Solution needs to be adapted to particular cases
 - E.g. user biometrics
- Data is cheap and abundant; knowledge is expensive and scarce



Applications of Machine Learning

From: cheapsales@buystufffromme.com
To: ang@cs.stanford.edu
Subject: Buy now!

Deal of the week! Buy now!
Rolex w4tchs - \$100
Medicine (any kind) - \$50
Also low cost M0rgages
available.

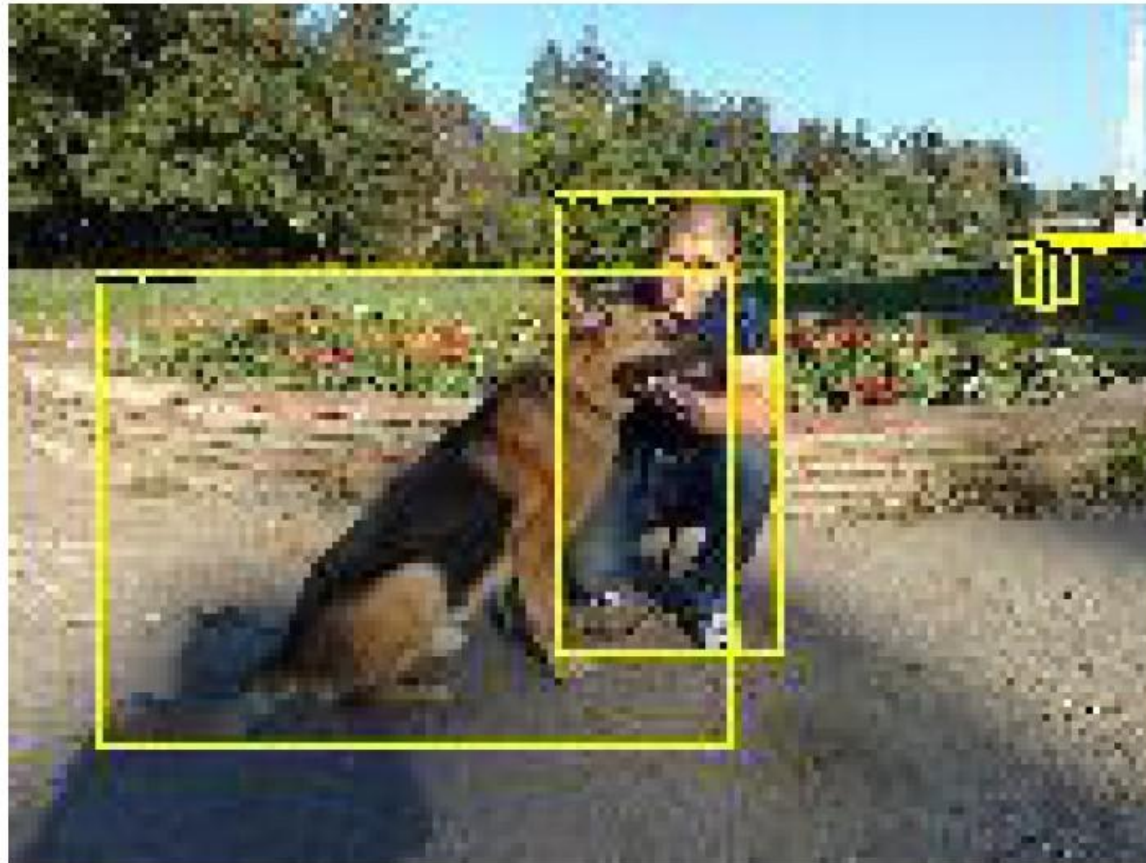
Spam

From: Alfred Ng
To: ang@cs.stanford.edu
Subject: Christmas dates?

Hey Andrew,
Was talking to Mom about plans
for Xmas. When do you get off
work. Meet Dec 22?
Alf

Non-spam

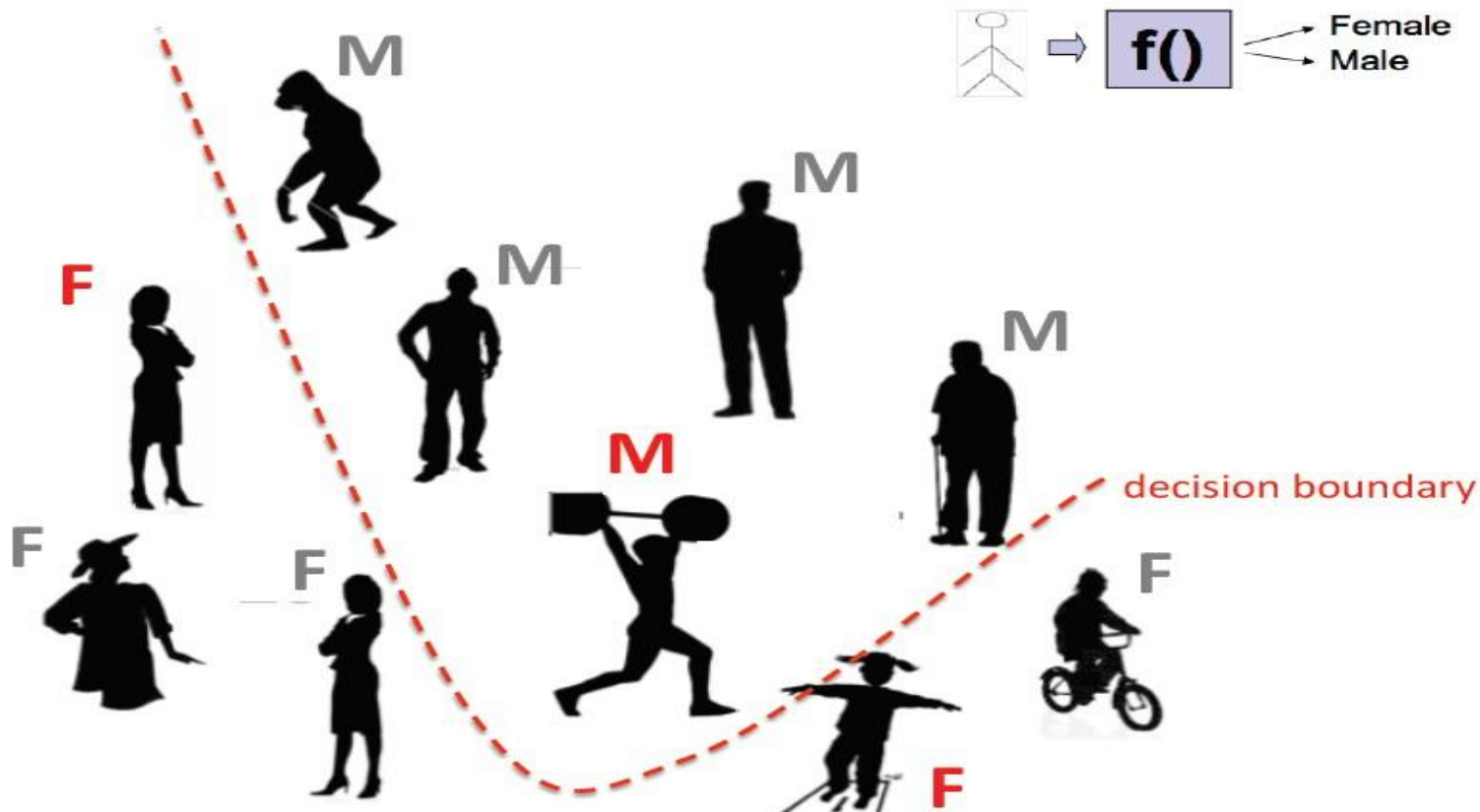
Applications of Machine Learning



Overview of ML

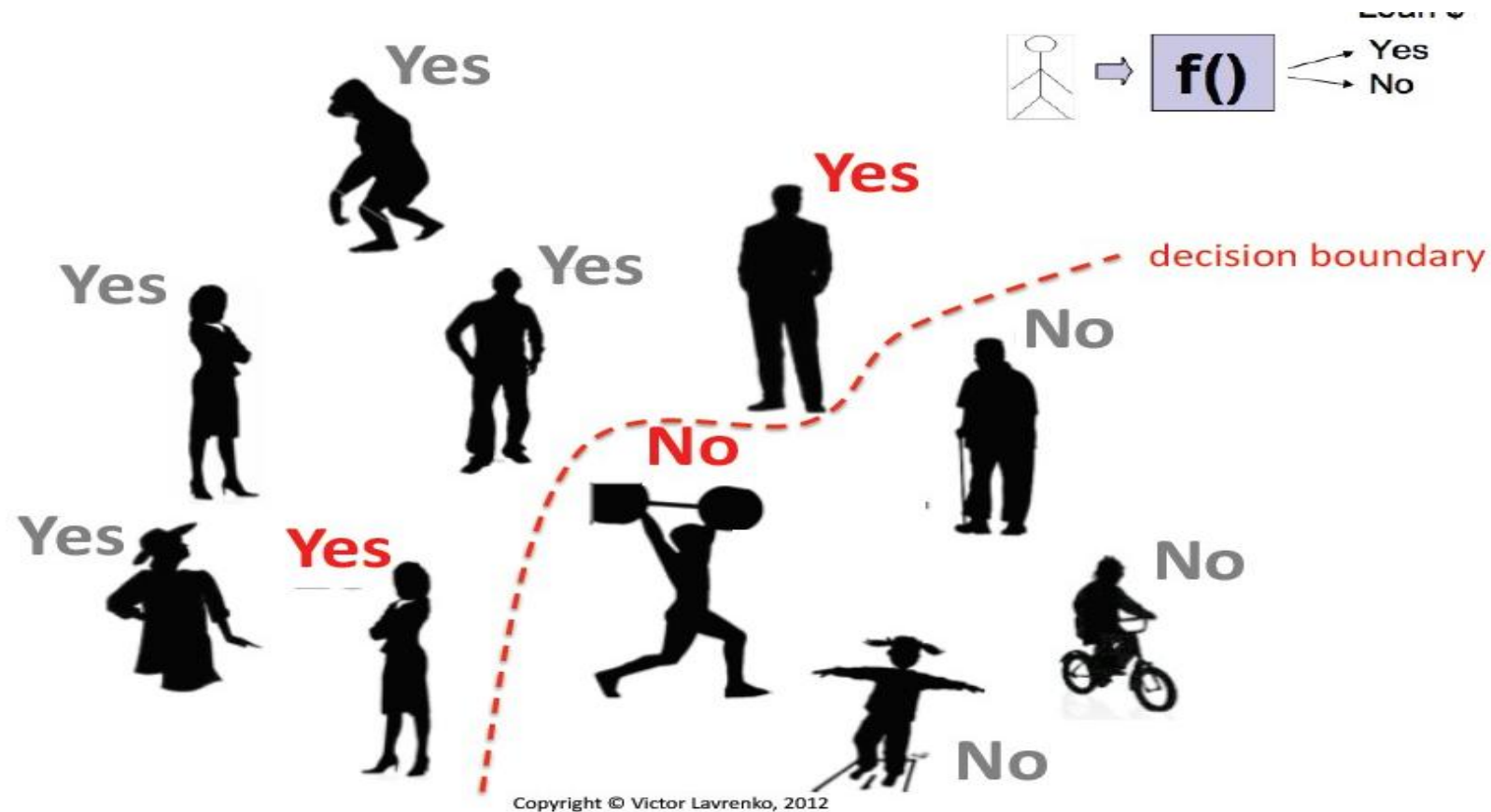
- Supervised learning
 - Predict an output y when given an input x
 - For categorical y : classification.
 - For real-valued y : regression.
- Unsupervised learning
 - Create an internal representation of the input, e.g. clustering, dimensionality reduction
 - This is important in ML as getting labels is often difficult and expensive
- Other settings of ML
 - Reinforcement learning (learning from “rewards”)
 - Semi-supervised learning (combines supervised + unsupervised)
 - Active learning, Transfer learning, Structured prediction

Classification (Supervised Learning)

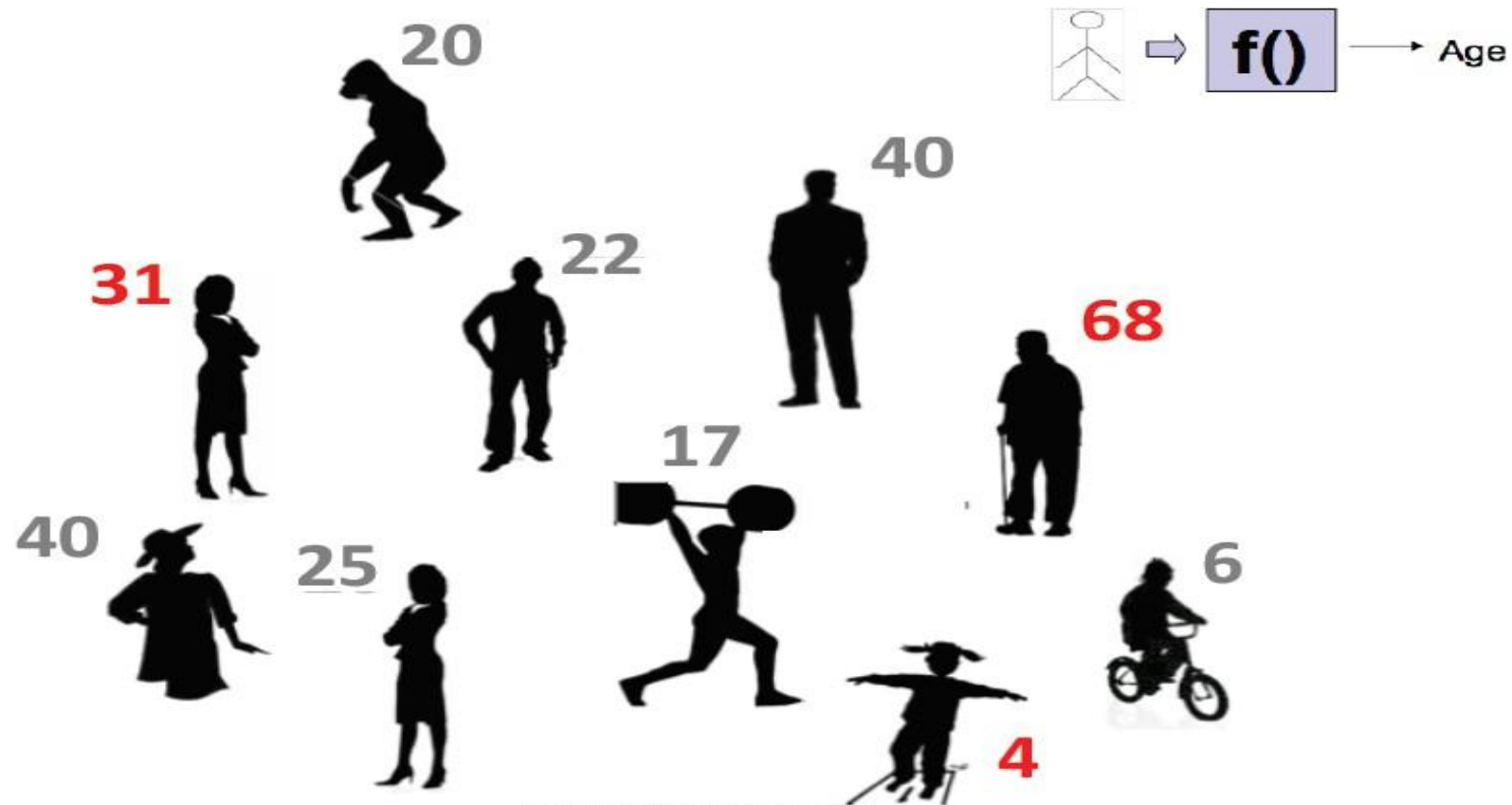


Copyright © Victor Lavrenko, 2012

Classification (Supervised Learning)

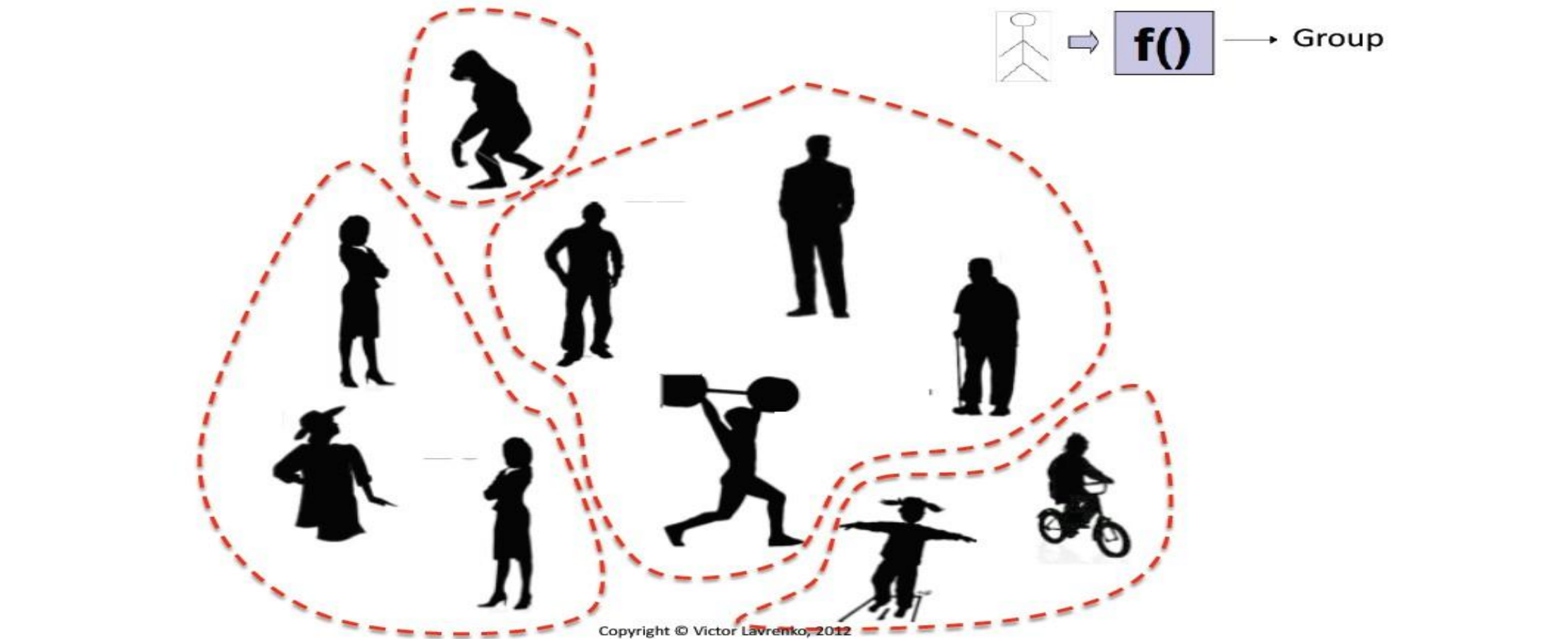


Regression (Supervised Learning)



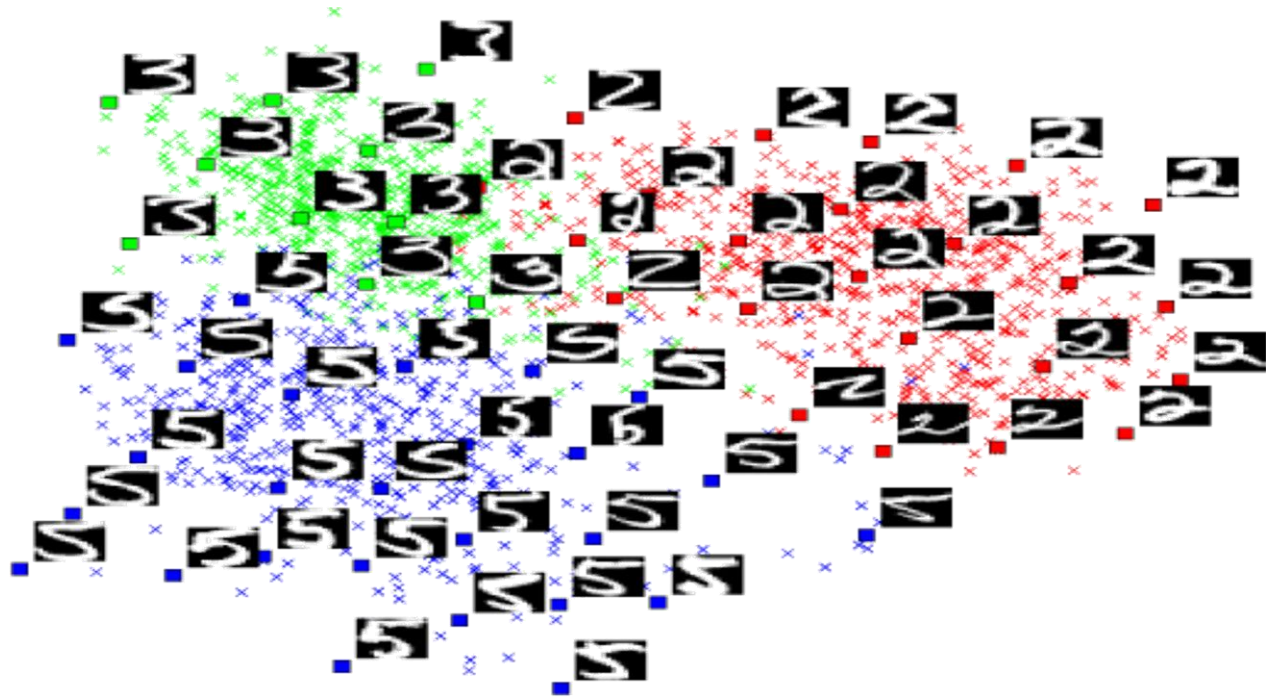
Copyright © Victor Lavrenko, 2012

Clustering (Unsupervised Learning)



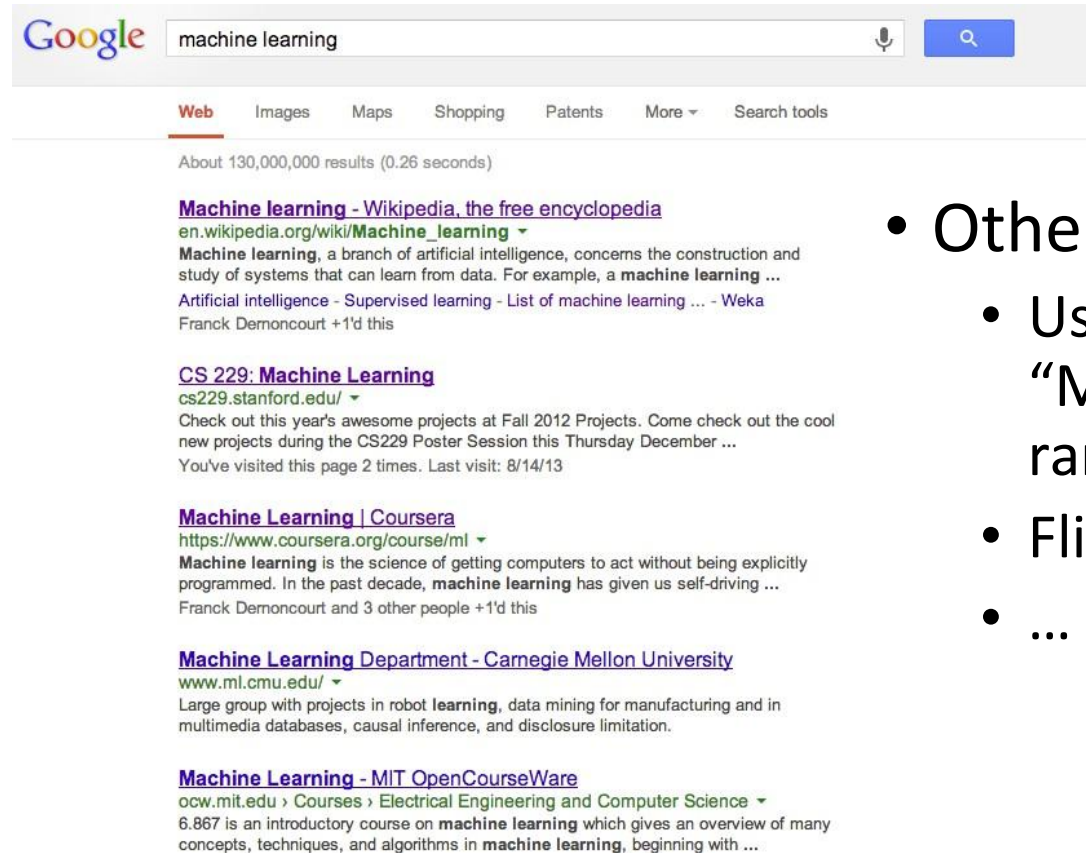
Dimensionality Reduction (Unsupervised Learning)

- Large sample size is required for high-dimensional data
- Query accuracy and efficiency degrade rapidly as the dimension increases
- Strategies
 - Feature reduction
 - Feature selection
 - Manifold learning
 - Kernel learning



Other Settings: Ranking (Supervised Learning)

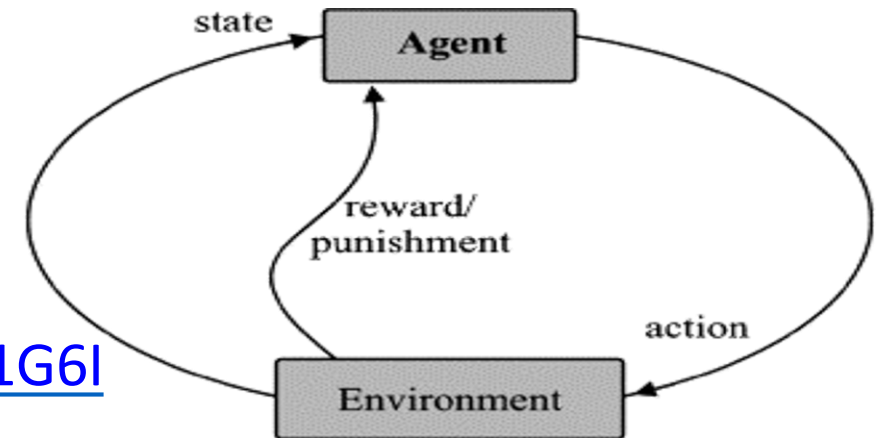
Given a query and
a set of web pages,
rank them according
to relevance



- Other applications
 - User preference, e.g. Netflix “My List” -- movie queue ranking
 - Flight search (search in general)
 - ...

Other Settings: Reinforcement Learning

- Learning a policy: A **sequence** of outputs
- No supervised output but delayed reward
 - E.g. Game playing
 - E.g. Robot in a maze
- Multiple agents, partial observability, ...
- Example (Simple Demo):
 - <https://www.youtube.com/watch?v=DCjbk4m1G6I>



ML Problems

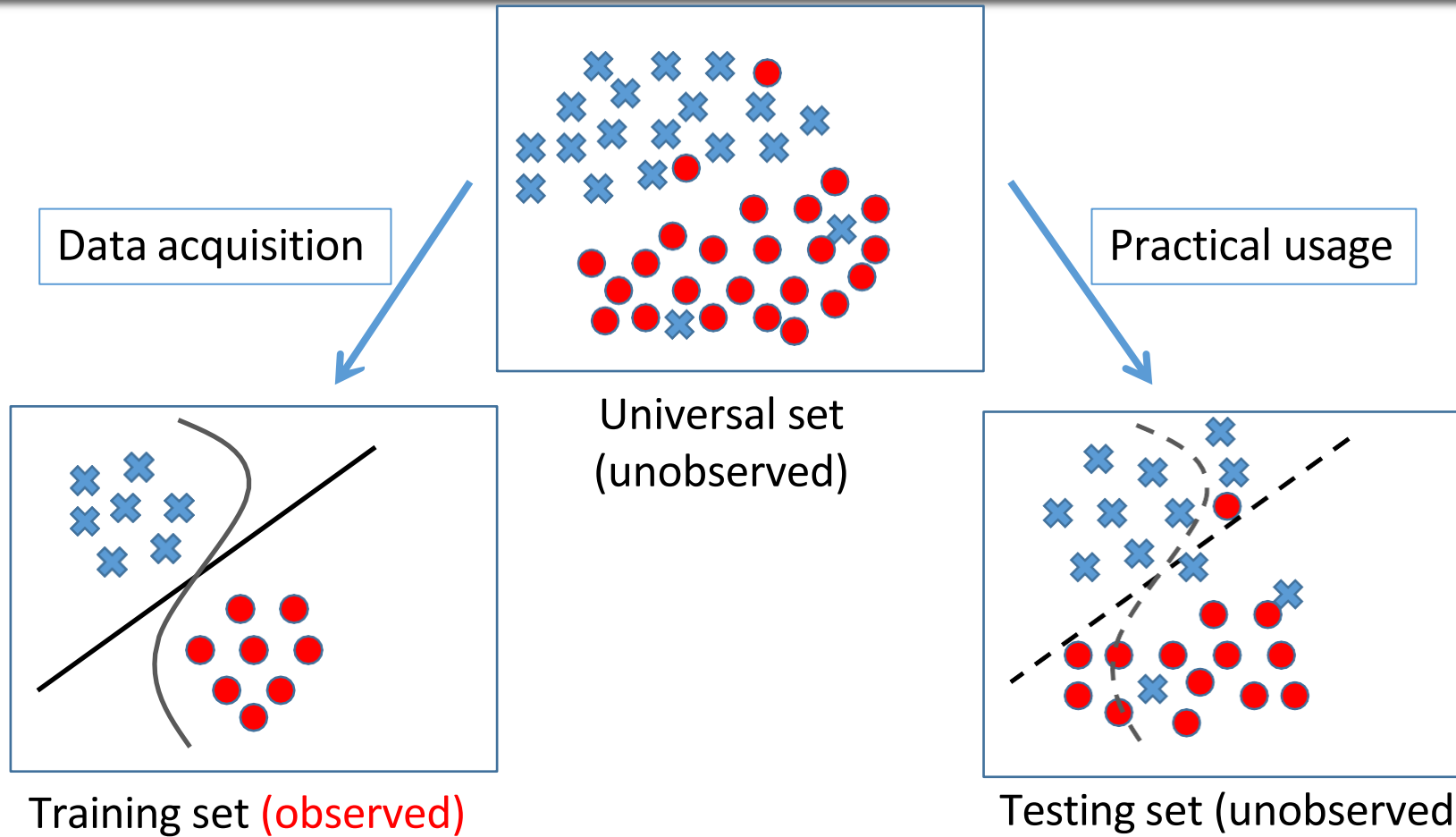
	<i>Supervised Learning</i>	<i>Unsupervised Learning</i>
<i>Discrete</i>	classification or categorization	clustering
<i>Continuous</i>	regression	dimensionality reduction

- In unsupervised learning, there are no labels y , but we can think of the algorithm's output structure as either discrete (clusters) or continuous (low dimensional representations)

ML in Practice

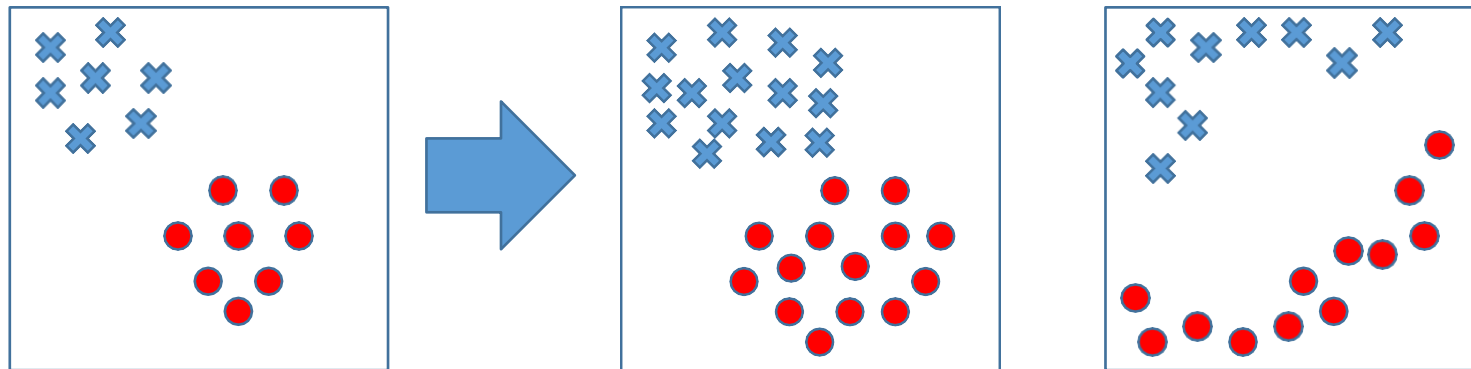
- Understanding domain, prior knowledge, and goals
- Data integration, selection, cleaning, pre-processing, etc.
- Learning models
- Interpreting results
- Consolidating and deploying discovered knowledge
- Loop

Training and Testing ML Models



Training and Testing ML Models

- Training is the process of making the system able to learn.
- Training set and testing set may not come from the same distribution
- We need to make some assumptions or bias



Types of Models

- Generative vs Discriminative Models
- Parametric vs Non-Parametric Models
- Inductive vs Transductive Learning
- Online vs Offline Learning

Training
Images



Training

Training
Labels

Image
Features

Classifier
Training

Trained
Classifier

Testing

Image
Features

Trained
Classifier

Prediction
Outdoor

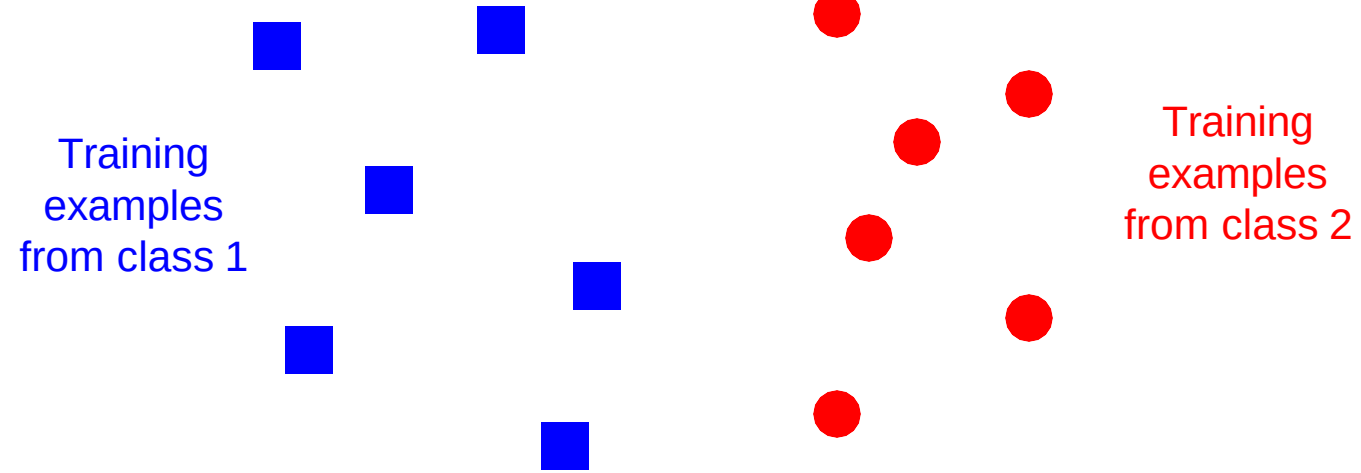


Test Image

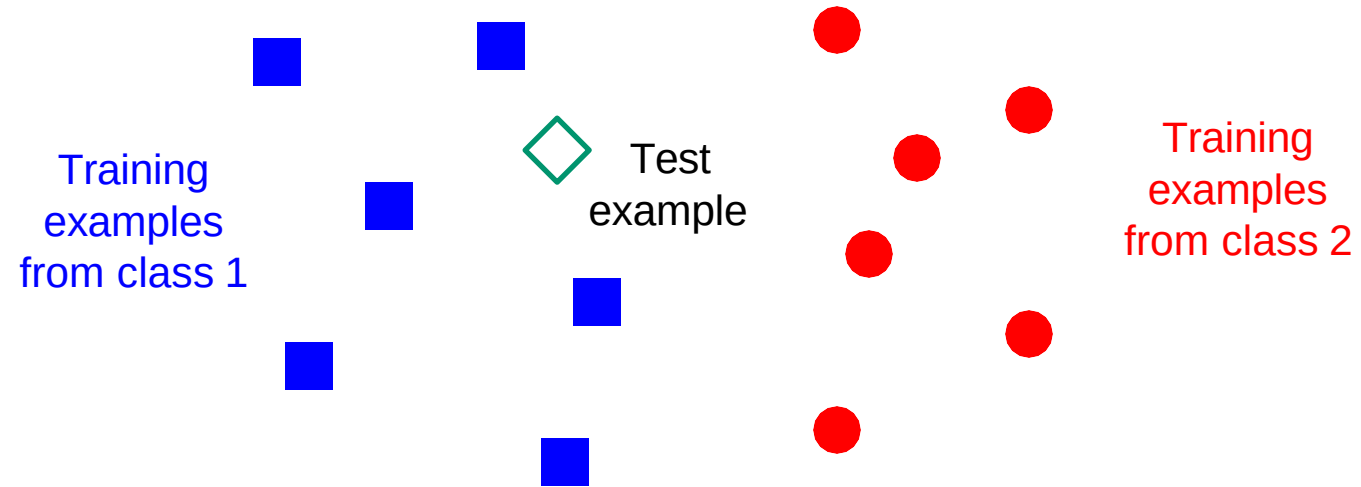
Classification Methods

- k-Nearest Neighbors
- Linear Classifier
- Decision Trees
- Naïve Bayes
- Support Vector Machines
- Logistic Regression
- Neural Networks
- Ensemble Methods (Boosting, Random Forests)

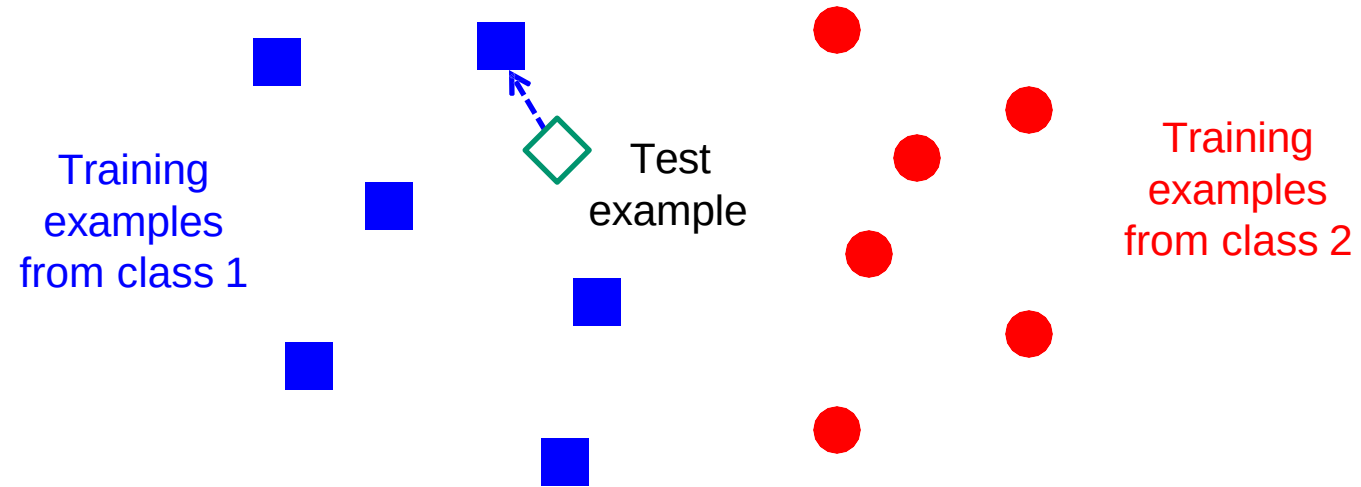
Classifiers: Nearest neighbor



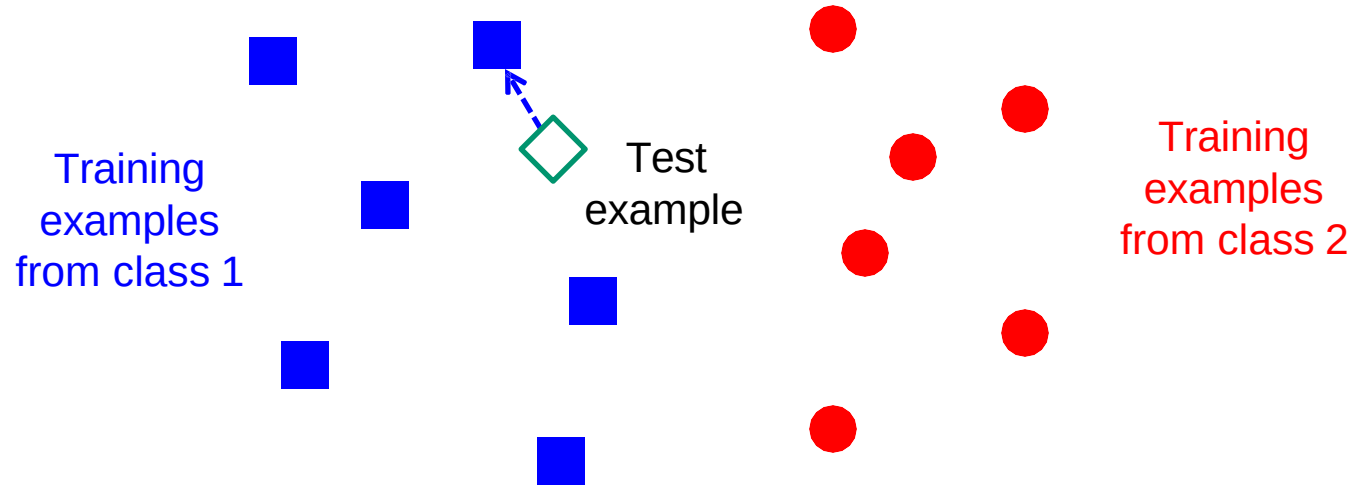
Classifiers: Nearest neighbor



Classifiers: Nearest neighbor



Classifiers: Nearest neighbor



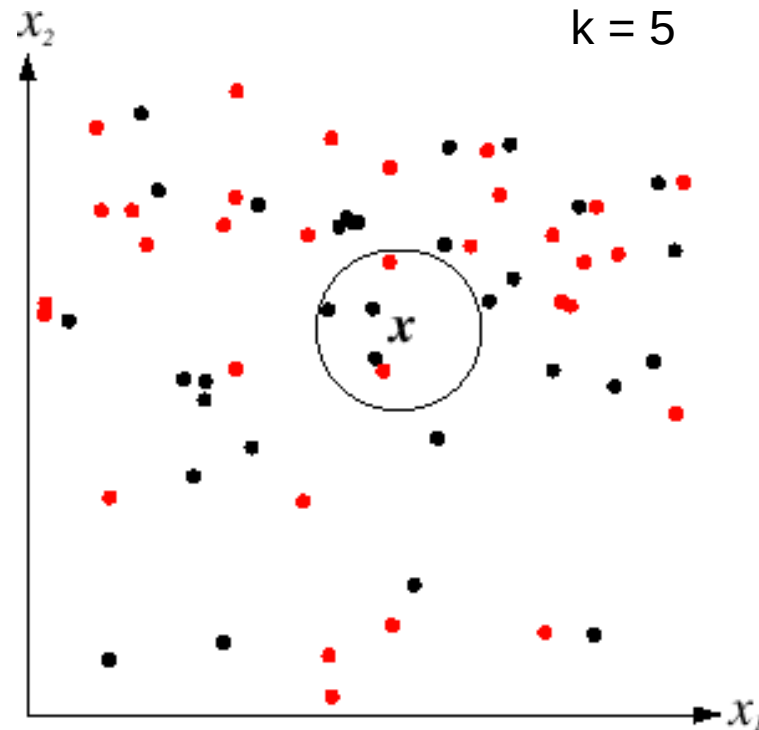
$f(\mathbf{x}) = \text{label of the training example nearest to } \mathbf{x}$

All we need is a distance function for our inputs

No training required!

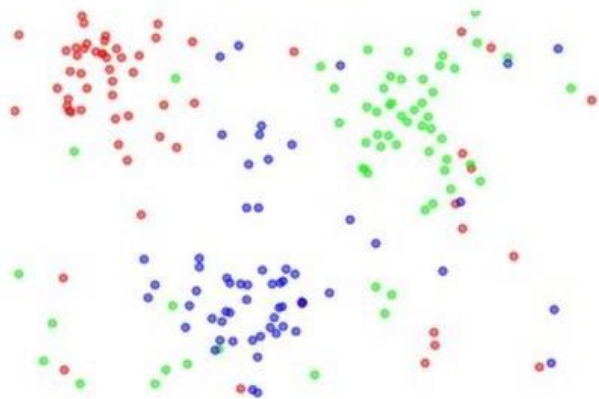
K-nearest neighbor classifier

- For a new point, find the k closest points from training data
- Vote for class label with labels of the k points, based on majority

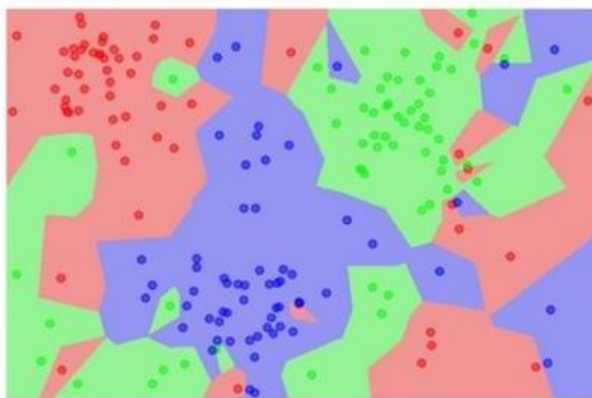


K-nearest neighbor classifier

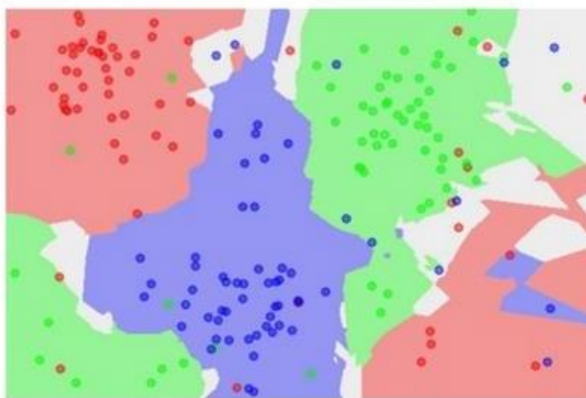
the data



NN classifier

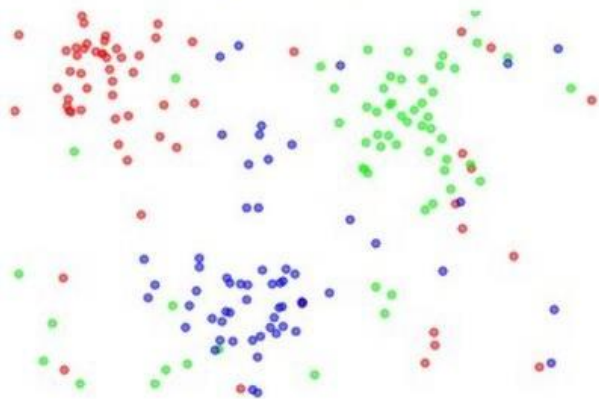


5-NN classifier

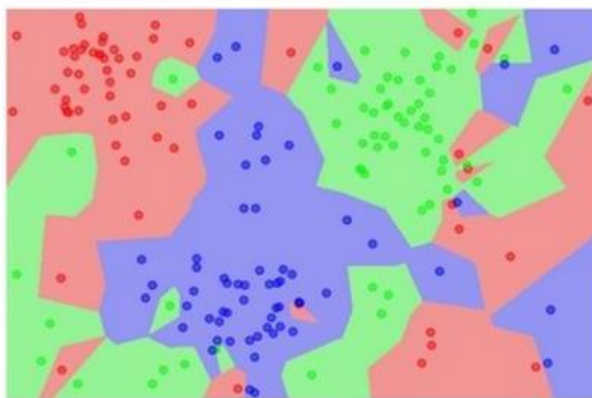


K-nearest neighbor classifier

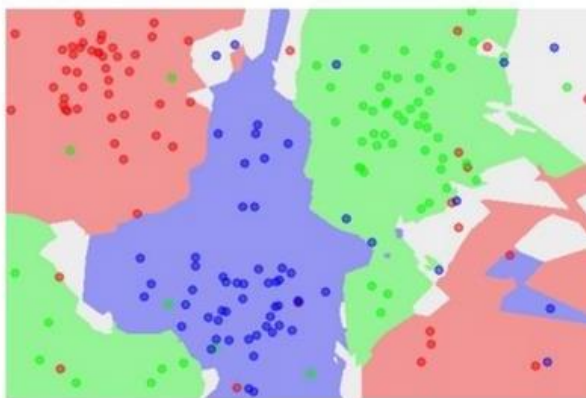
the data



NN classifier

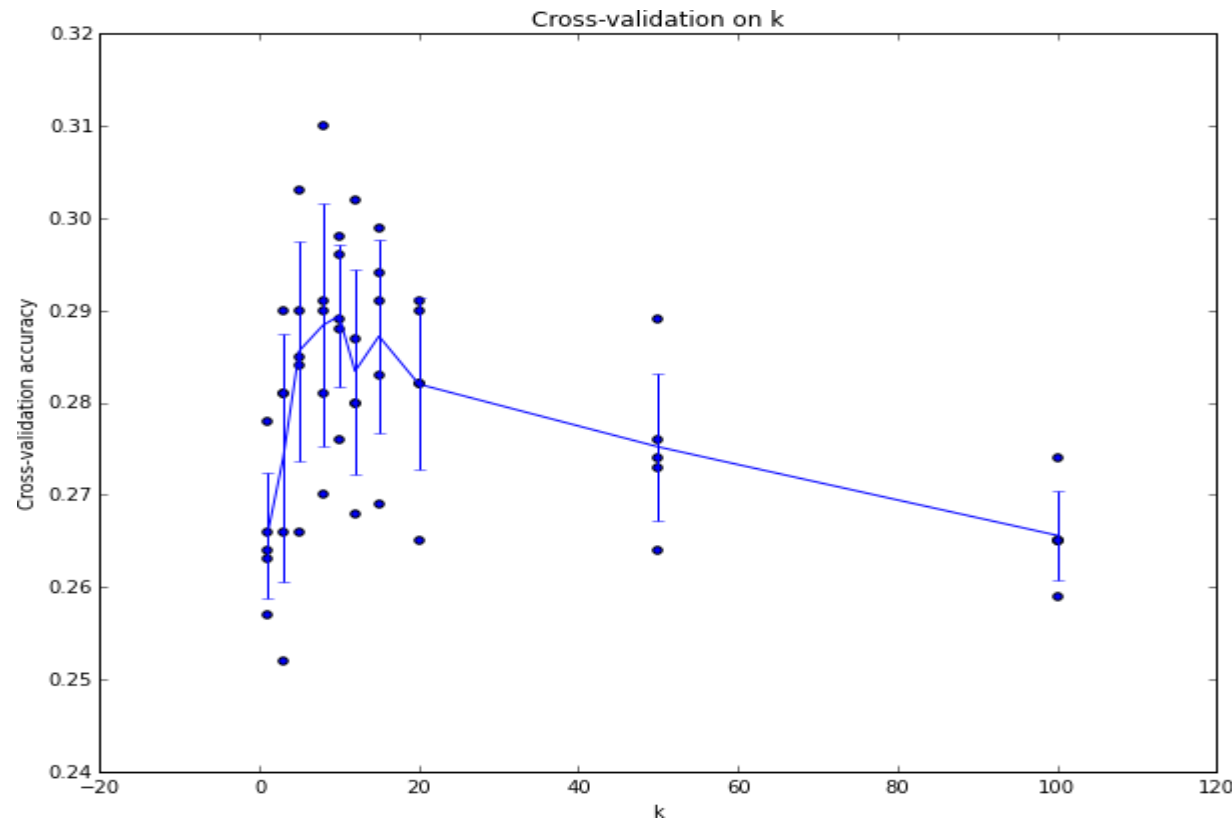


5-NN classifier



Which classifier is more robust to *outliers*?

Choice of K in KNN classifier



Example of a 5-fold cross-validation run for the parameter k . Note that in this particular case, the cross-validation suggests that a value of about $k = 7$ works best on this particular dataset (corresponding to the peak in the plot).

Classifiers: K-Nearest neighbor

“Non-parametric” classifier: the entire training set is essentially the model parameters.

Classifiers: K-Nearest neighbor

“Non-parametric” classifier: the entire training set is essentially the model parameters.

Pros:

- **Very fast at training** time
- **Flexible**: all it requires is a way to compute similarity or distances between pairs of features. Applies to many different kinds of features.
- Works with **any number of classes**.
- Works well in practice for large datasets (but see cons)

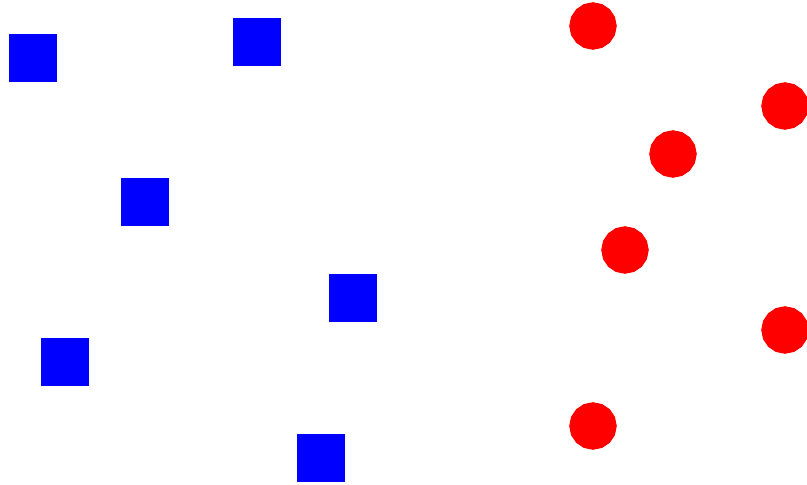
Classifiers: K-Nearest neighbor

“Non-parametric” classifier: the entire training set is essentially the model parameters.

Cons:

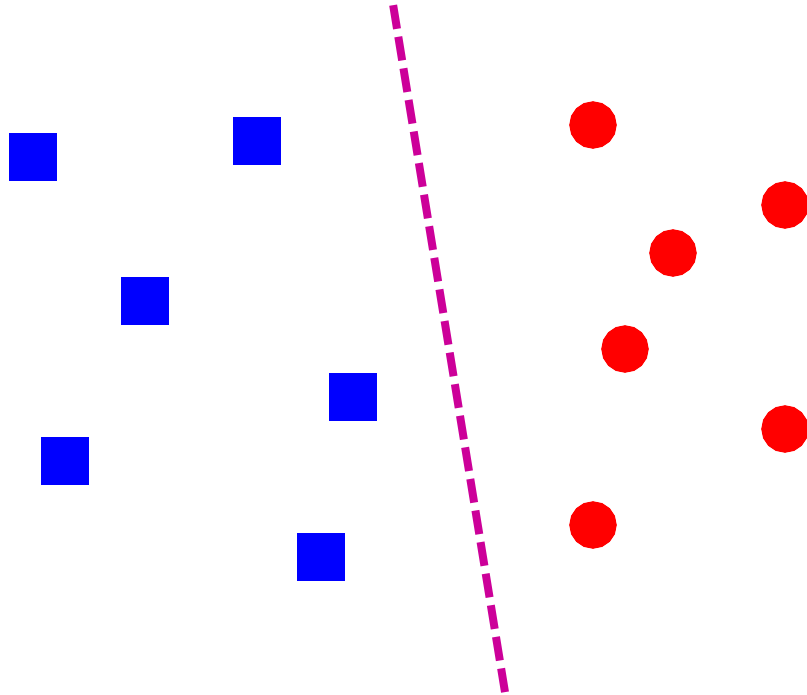
- The classifier must ***remember all of the training data*** and store it for future comparisons with the test data.
- This is **space inefficient** because datasets may easily be gigabytes in size.
- **Slow at test time** (need to compute distances between test example and every training example)
- Optimum value of **K is not known**.

Linear classifiers – 2 class problem



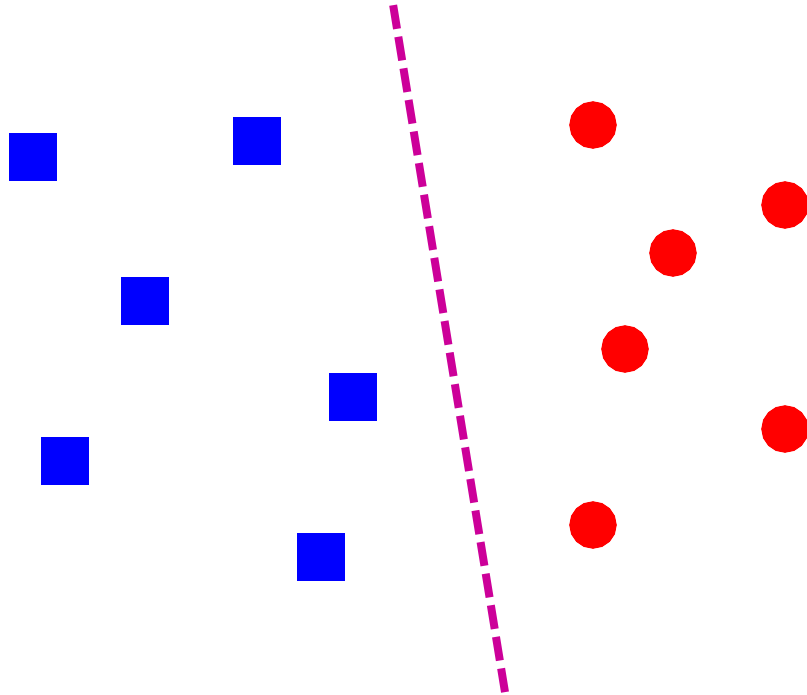
Find a *linear function* to separate the classes:

Linear classifiers – 2 class problem



Find a *linear function* to separate the classes:

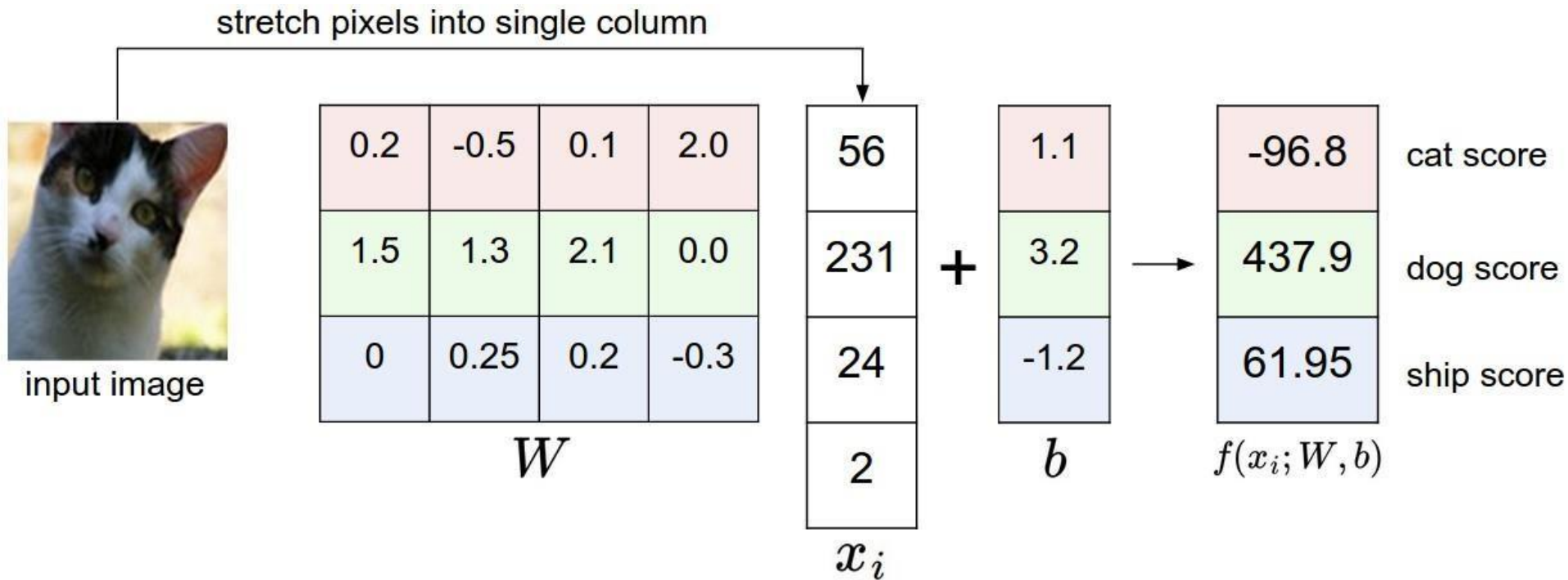
Linear classifiers – 2 class problem



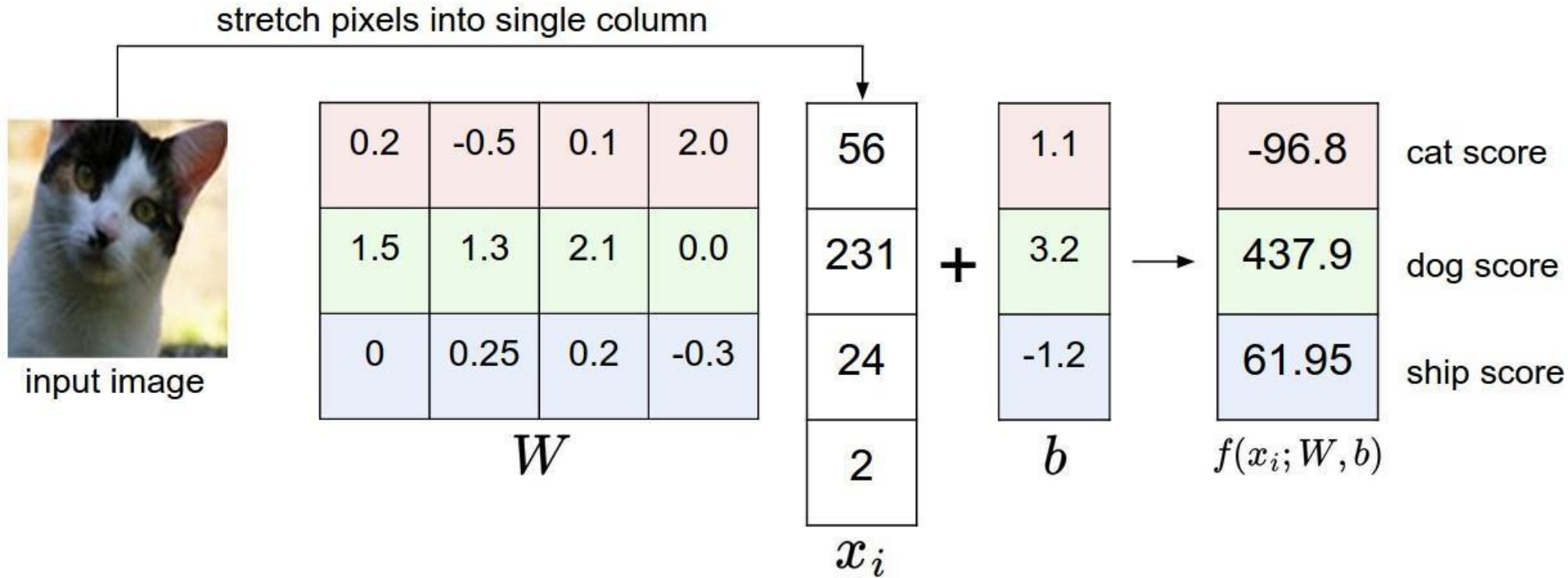
Find a *linear function* to separate the classes:

$$f(\mathbf{x}) = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b)$$

Linear classifiers – more than 2 class

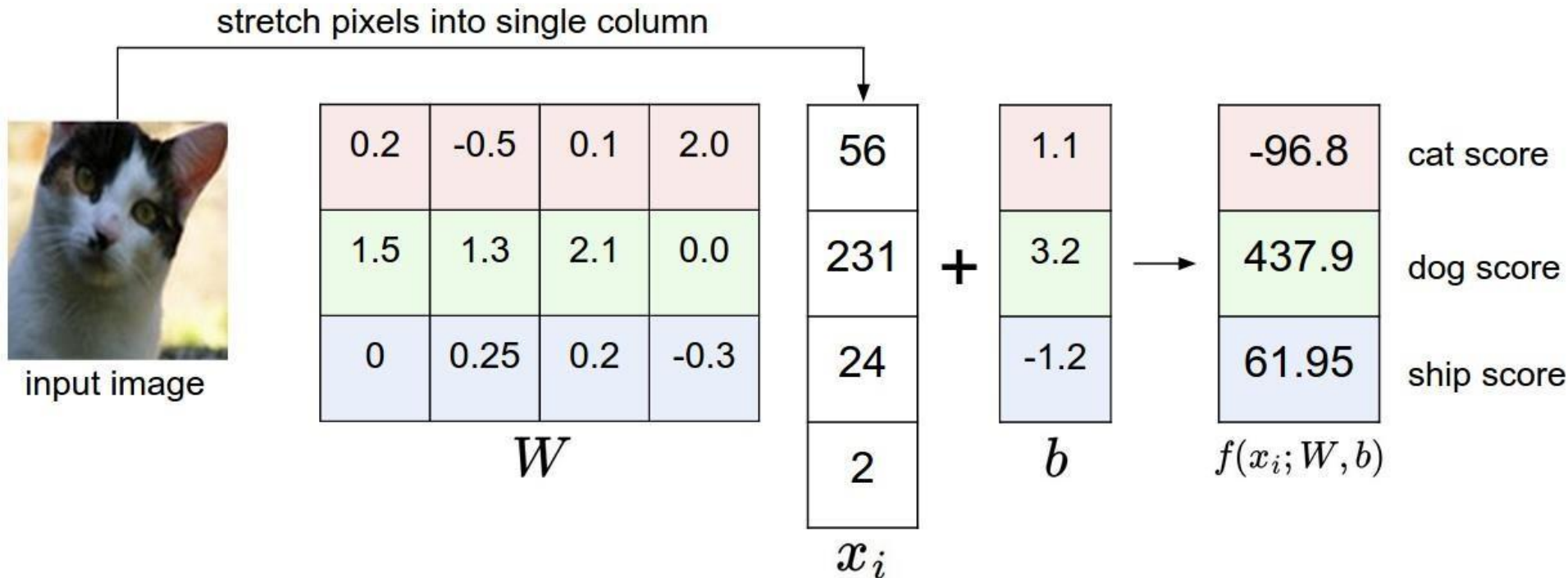


Linear classifiers – more than 2 class



$$f(x_i, W, b) = Wx_i + b$$

Linear classifiers – more than 2 class



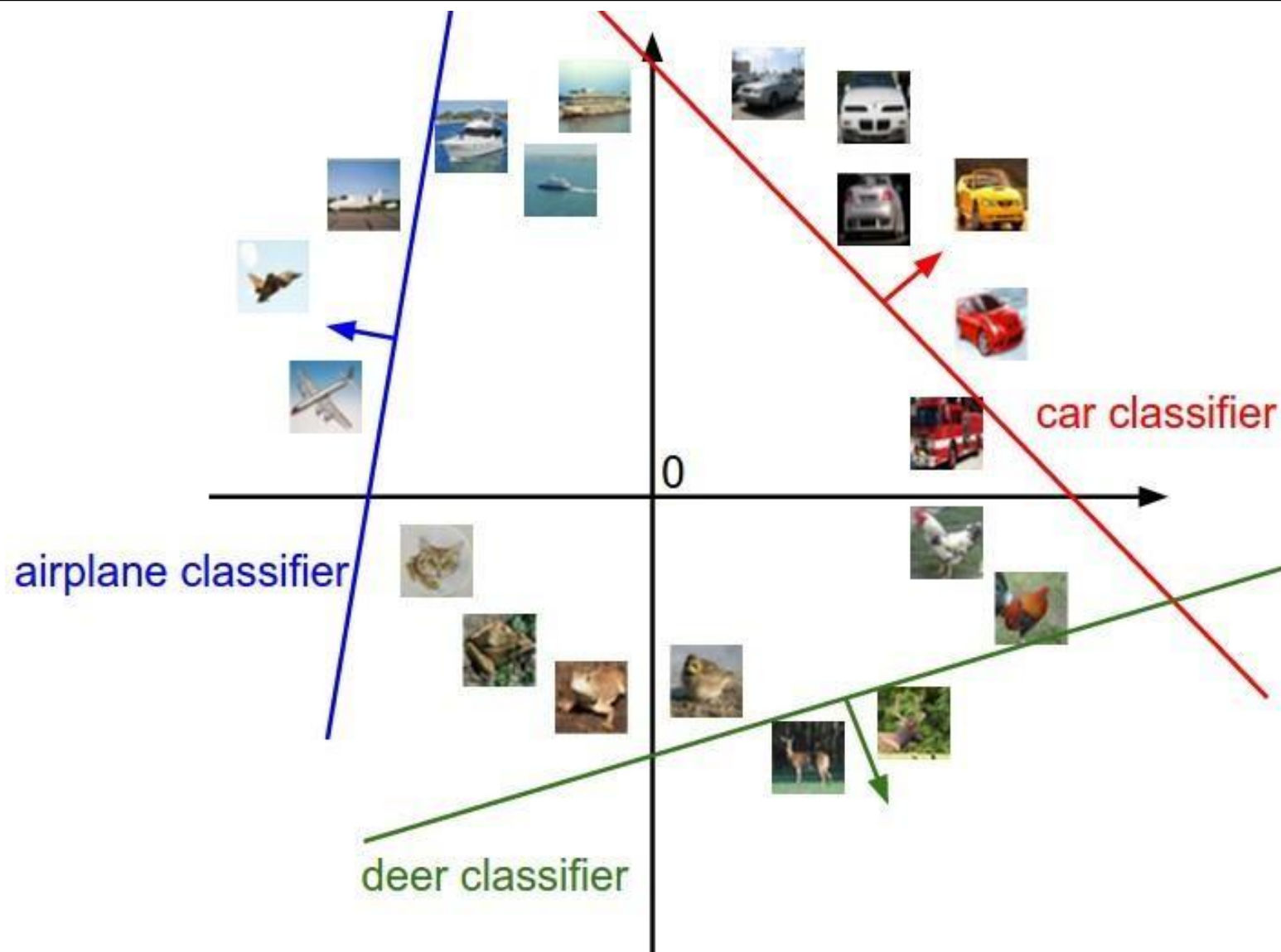
$$f(x_i, W, b) = Wx_i + b$$

Image x_i has all of its pixels flattened out to a single column vector of shape $[D \times 1]$.

Matrix W (of size $[K \times D]$), and vector b (of size $[K \times 1]$) are the **parameters**.

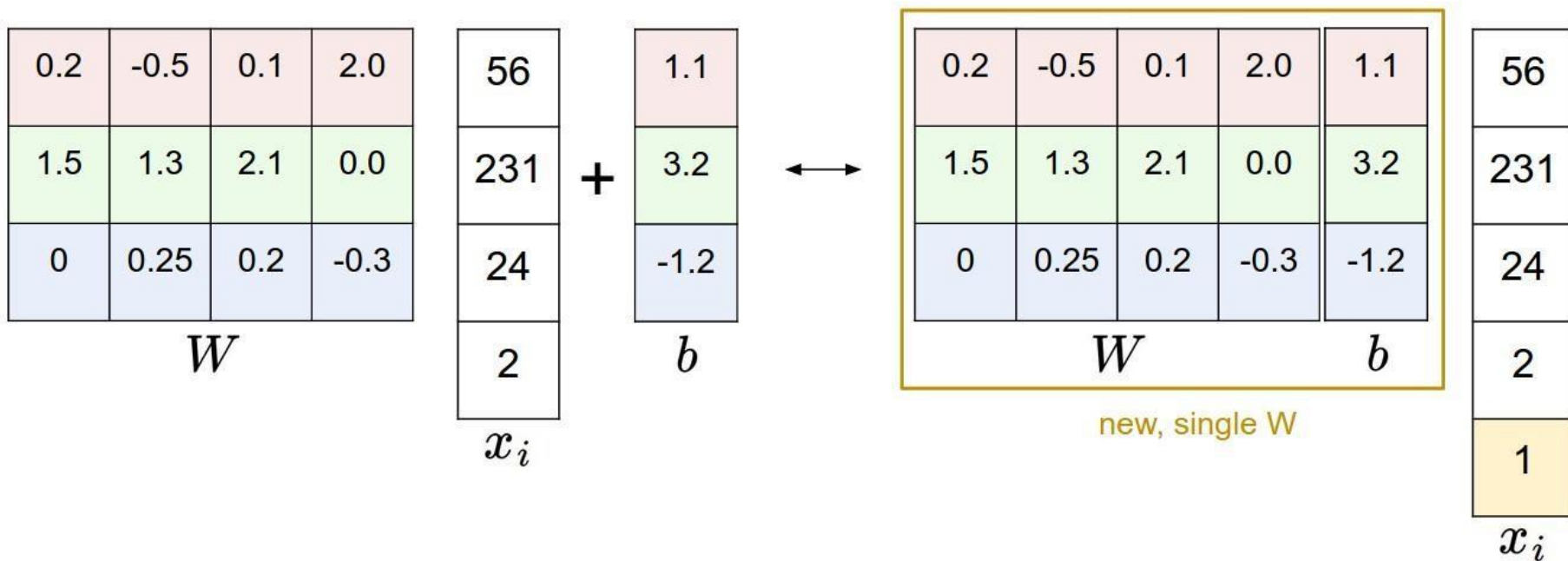
K is the number of classes.

Analogy of images as high-dimensional points



Source: cs231n, <http://cs231n.github.io/linear-classify/>

Bias Trick



Linear classifiers

“Parametric” classifier: model defined by a small number of parameters (w , b)

Pros:

- Very fast at test time

Cons:

- Slow at training time: need to estimate the parameters
- Data may not be linearly separable

Nearest neighbor vs. linear classifiers

- NN pros:
 - Simple to implement
 - Decision boundaries not necessarily linear
 - Works for any number of classes
 - *Nonparametric* method

Nearest neighbor vs. linear classifiers

- NN pros:
 - Simple to implement
 - Decision boundaries not necessarily linear
 - Works for any number of classes
 - *Nonparametric* method
- NN cons:
 - Need good distance function
 - Slow at test time, Memory in-efficient

Nearest neighbor vs. linear classifiers

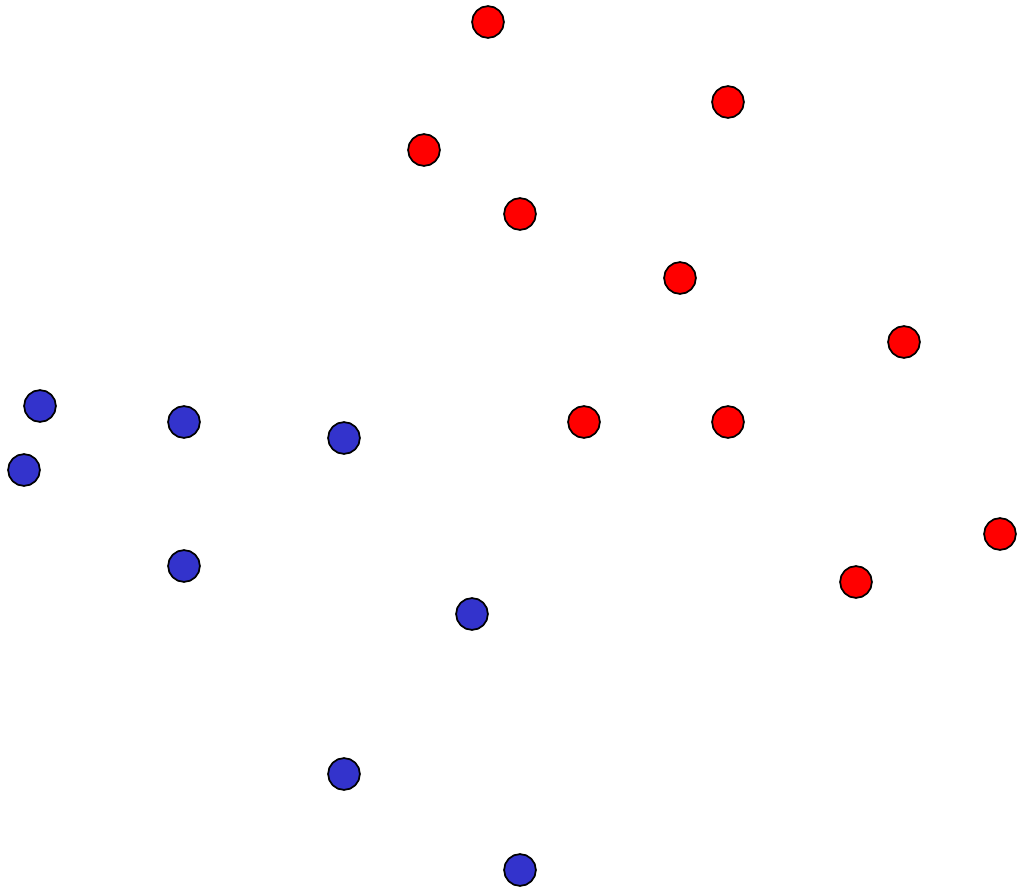
- NN pros:
 - Simple to implement
 - Decision boundaries not necessarily linear
 - Works for any number of classes
 - *Nonparametric* method
- NN cons:
 - Need good distance function
 - Slow at test time, Memory in-efficient
- Linear pros:
 - Low-dimensional *parametric* representation
 - Very fast at test time

Nearest neighbor vs. linear classifiers

- NN pros:
 - Simple to implement
 - Decision boundaries not necessarily linear
 - Works for any number of classes
 - *Nonparametric* method
- NN cons:
 - Need good distance function
 - Slow at test time, Memory in-efficient
- Linear pros:
 - Low-dimensional *parametric* representation
 - Very fast at test time
- Linear cons:
 - How to train the linear function?
 - What if data is not linearly separable?

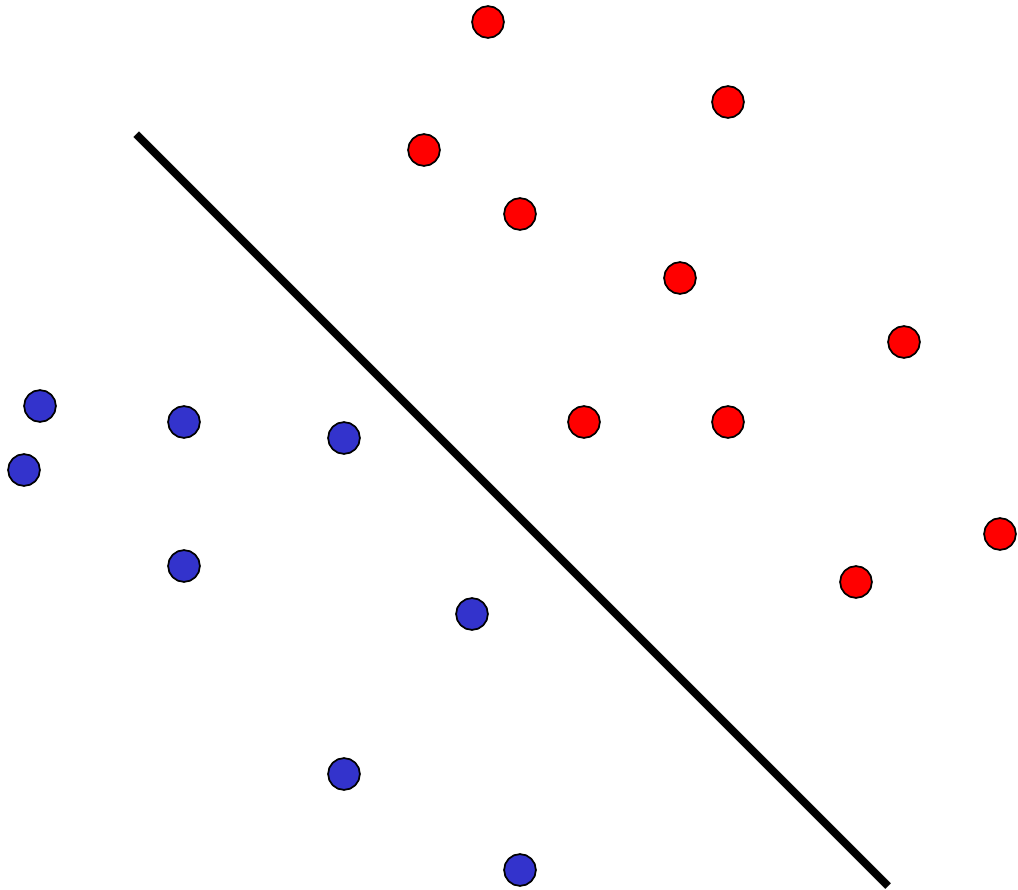
Support vector machines

- When the data is linearly separable, there may be more than one separator (hyperplane)



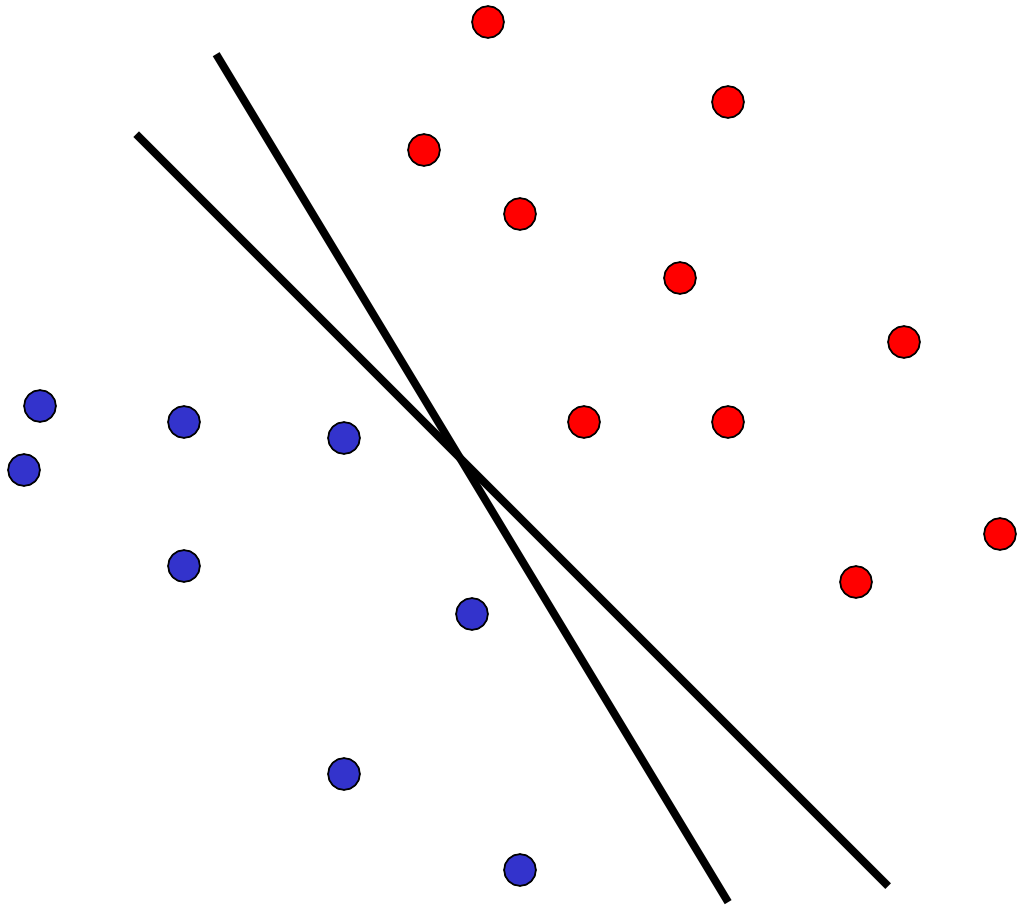
Support vector machines

- When the data is linearly separable, there may be more than one separator (hyperplane)



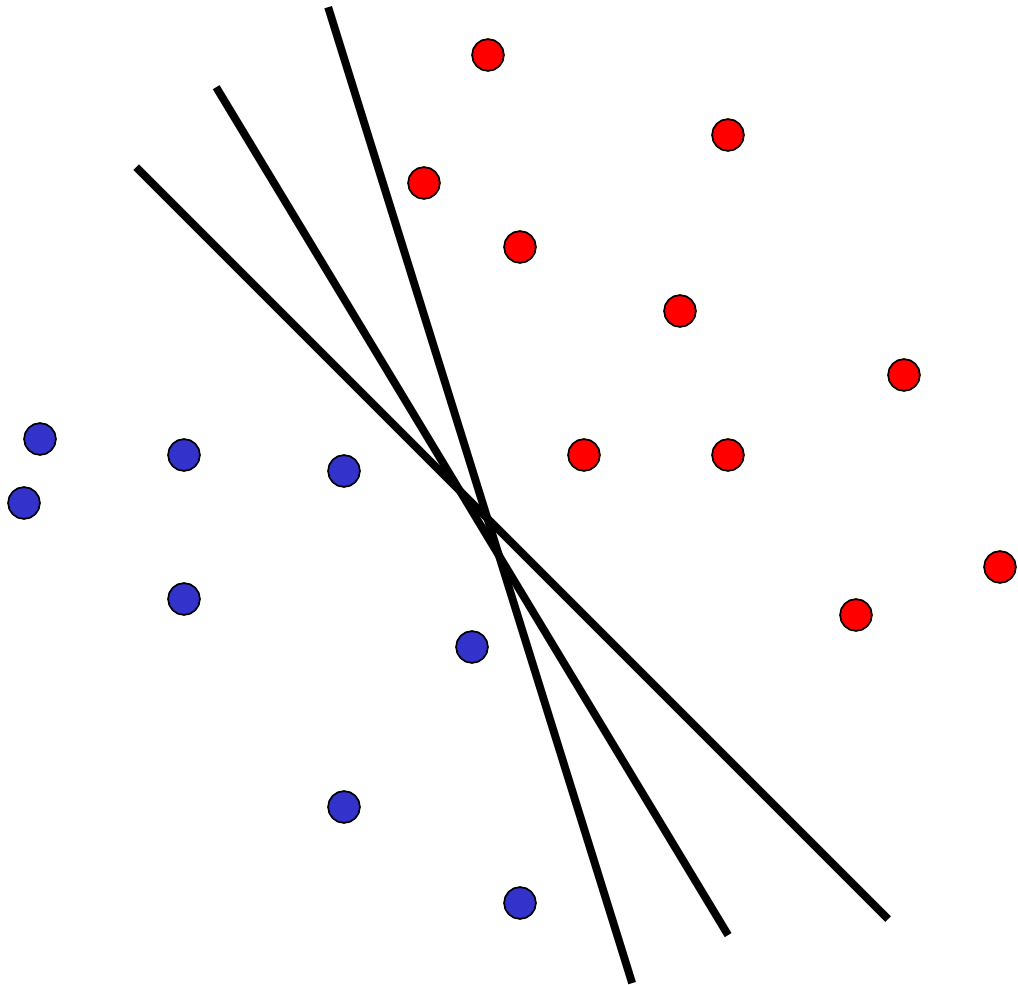
Support vector machines

- When the data is linearly separable, there may be more than one separator (hyperplane)



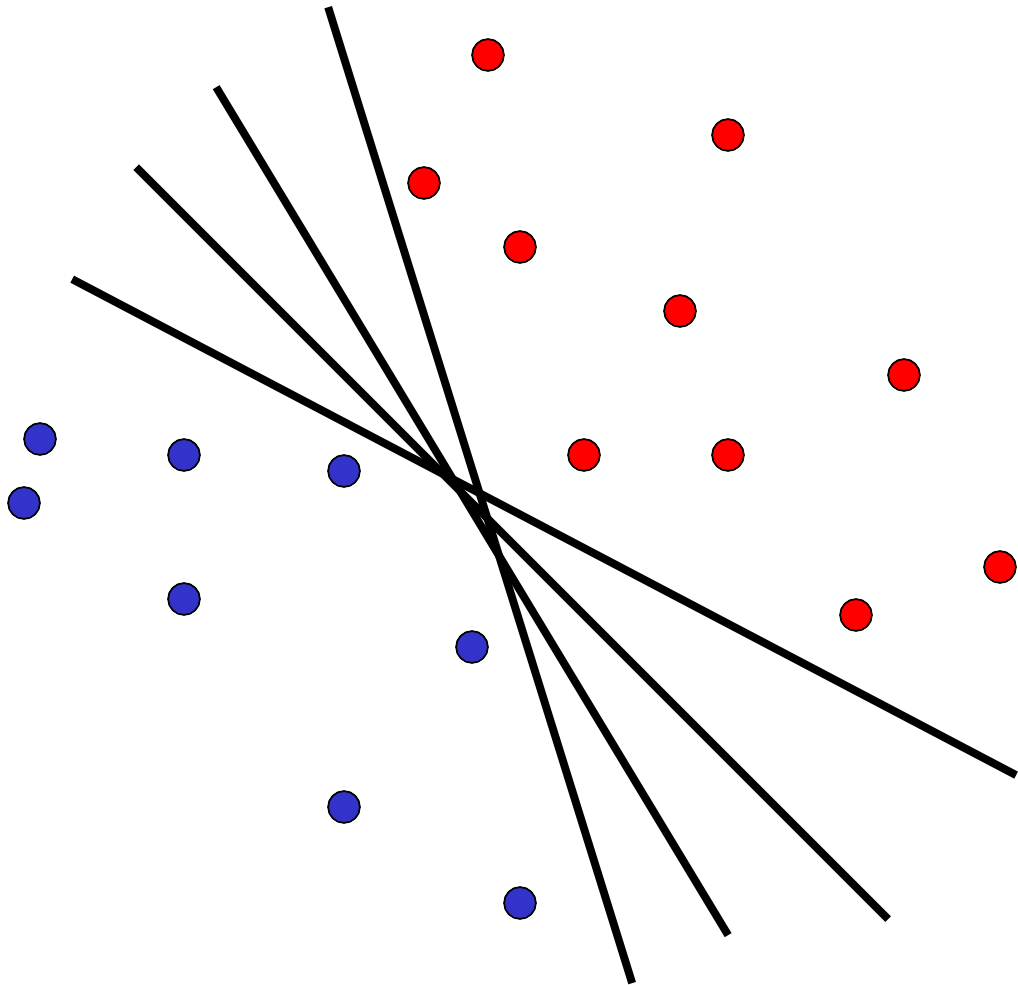
Support vector machines

- When the data is linearly separable, there may be more than one separator (hyperplane)



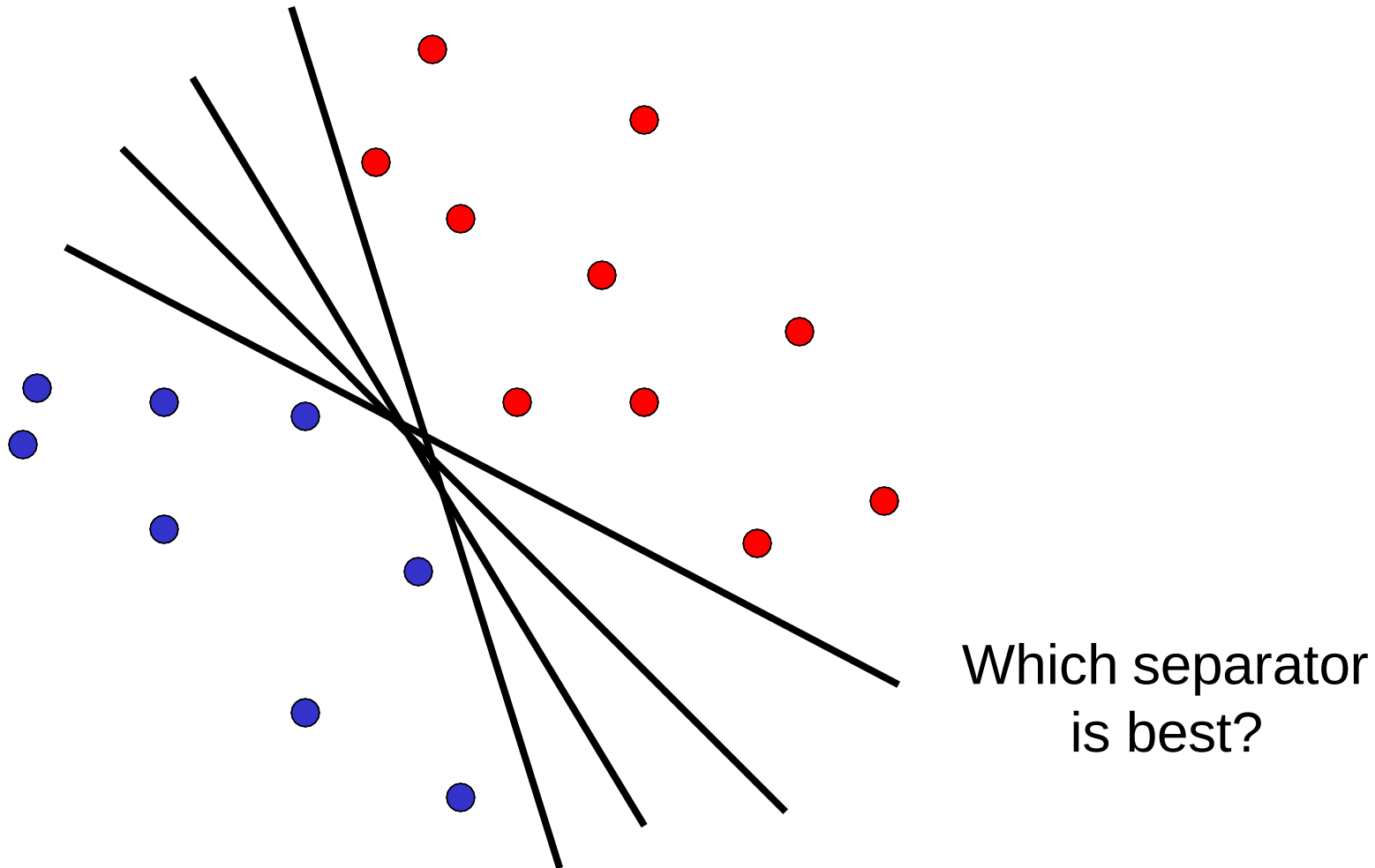
Support vector machines

- When the data is linearly separable, there may be more than one separator (hyperplane)



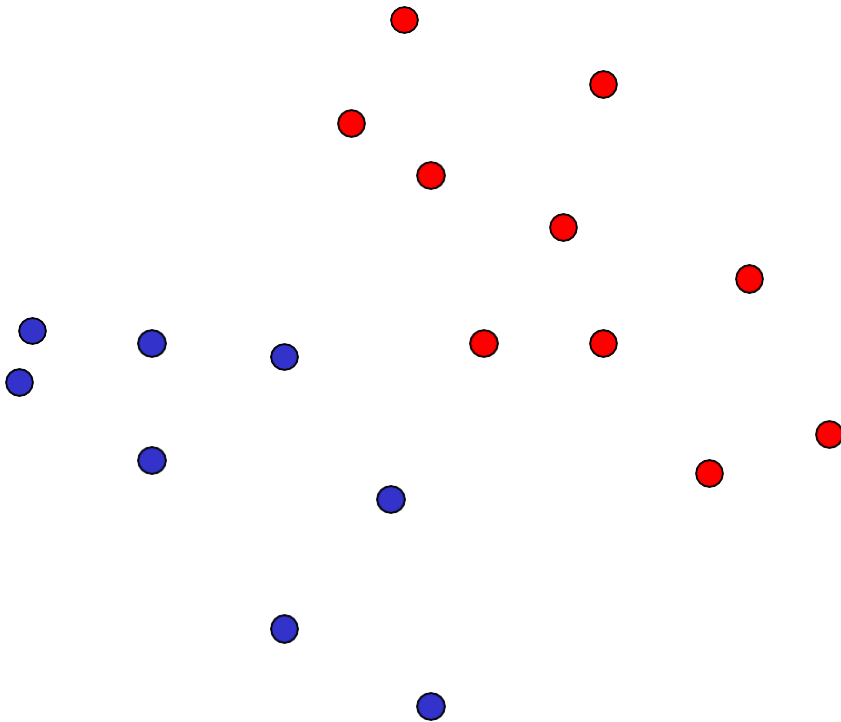
Support vector machines

- When the data is linearly separable, there may be more than one separator (hyperplane)



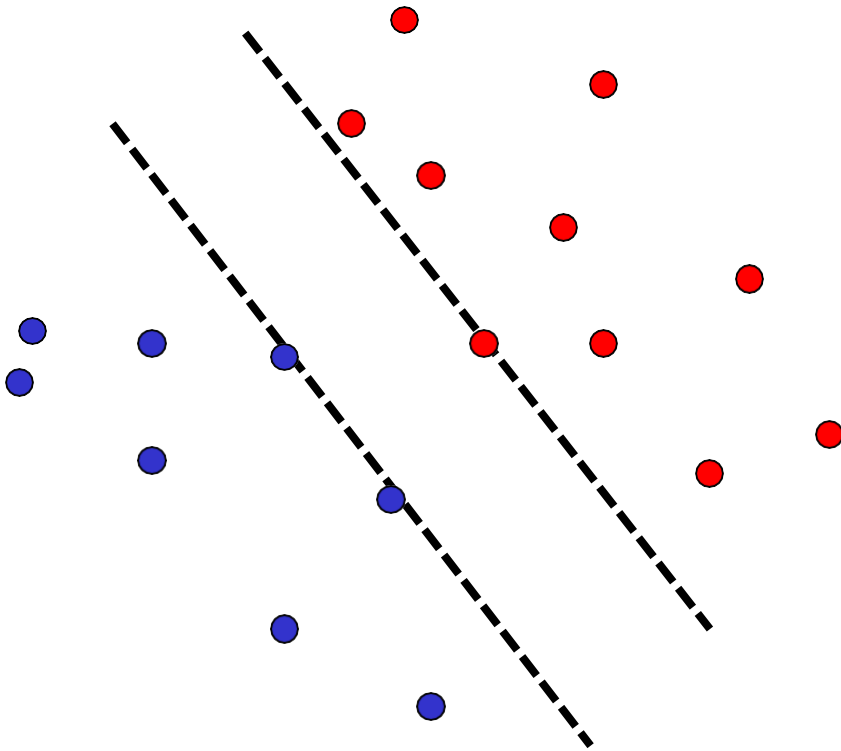
Support vector machines

- Find hyperplane that maximizes the *margin* between the positive and negative examples



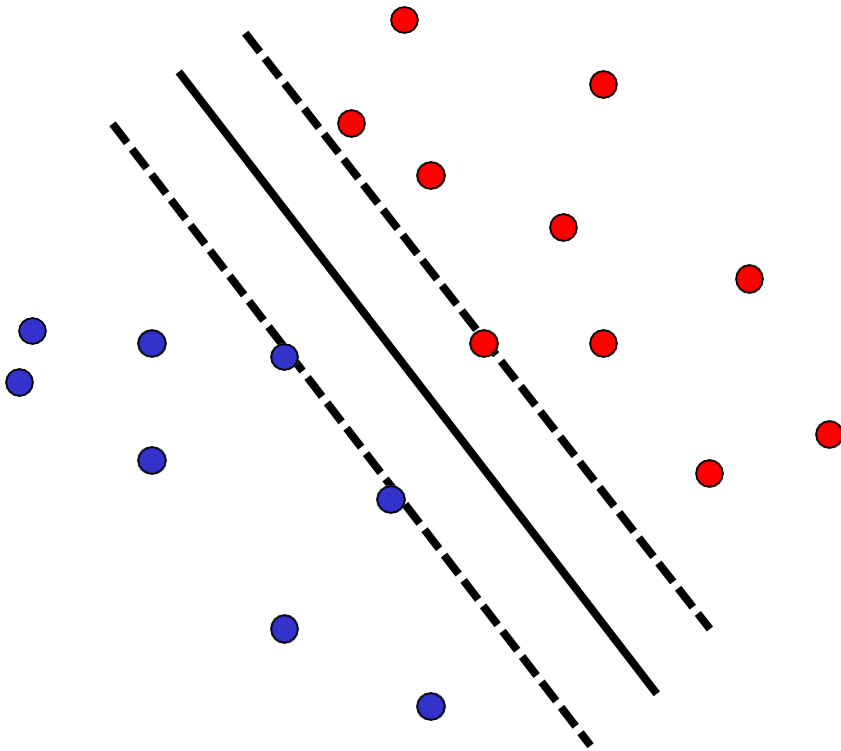
Support vector machines

- Find hyperplane that maximizes the *margin* between the positive and negative examples



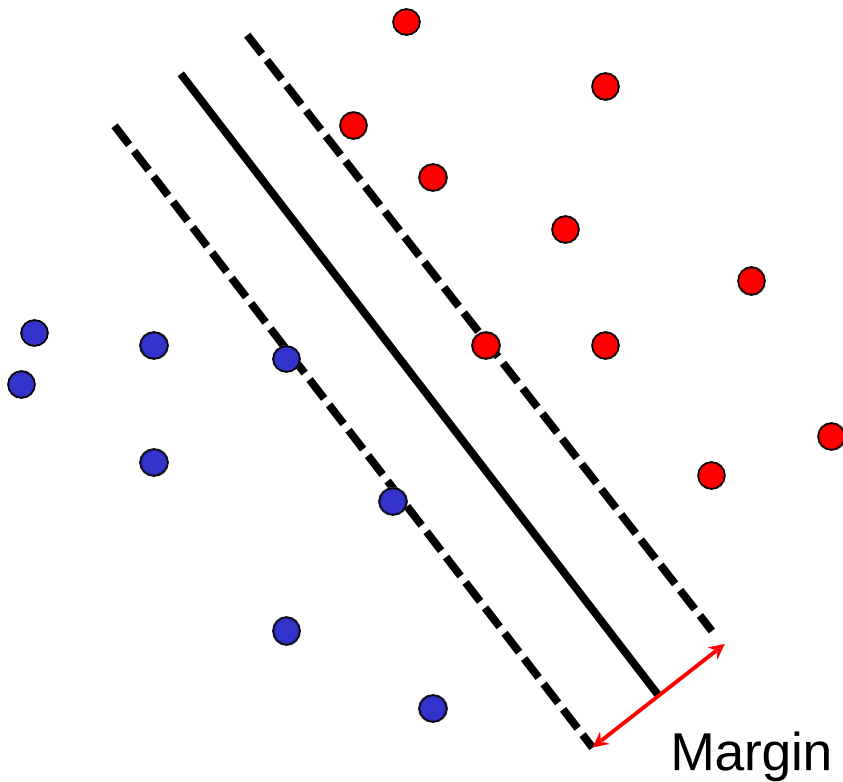
Support vector machines

- Find hyperplane that maximizes the *margin* between the positive and negative examples



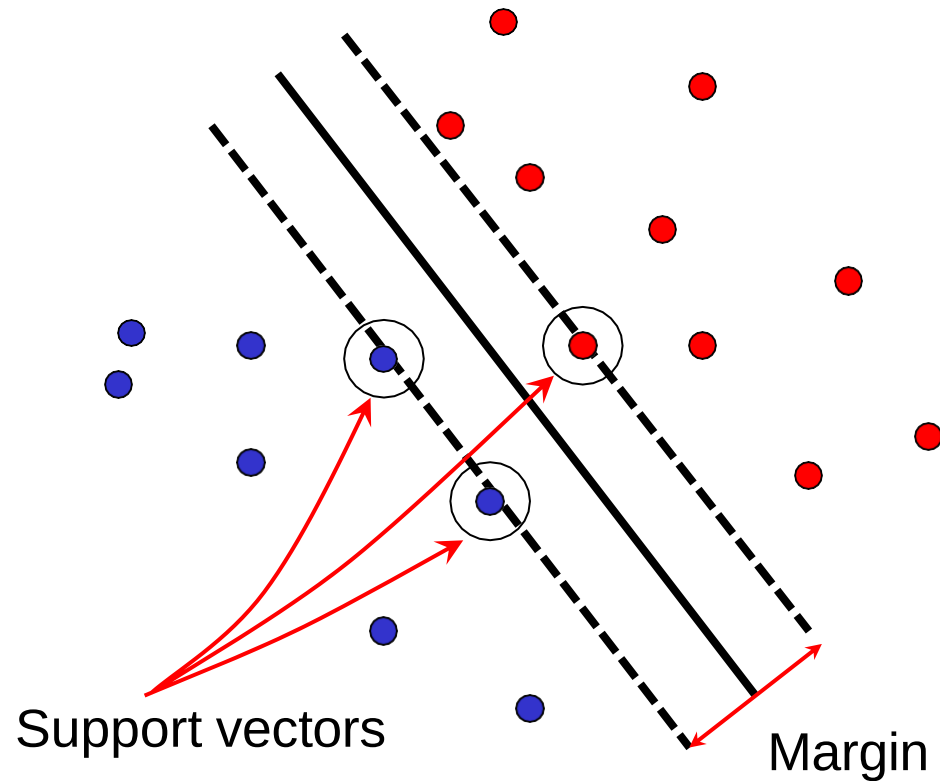
Support vector machines

- Find hyperplane that maximizes the *margin* between the positive and negative examples



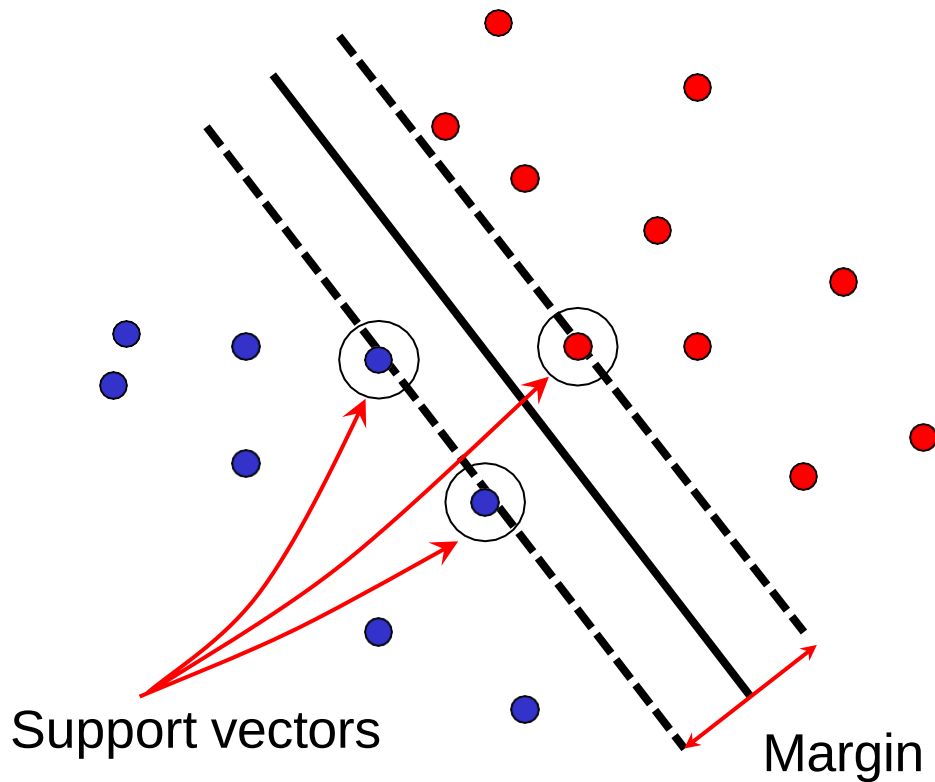
Support vector machines

- Find hyperplane that maximizes the *margin* between the positive and negative examples



Support vector machines

- Find hyperplane that maximizes the *margin* between the positive and negative examples



$$\mathbf{x}_i \text{ positive } (y_i = 1): \quad \mathbf{x}_i \cdot \mathbf{w} + b \geq 1$$

$$\mathbf{x}_i \text{ negative } (y_i = -1): \quad \mathbf{x}_i \cdot \mathbf{w} + b \leq -1$$

$$\text{Hyperplane: } \mathbf{x} \cdot \mathbf{w} + b = 1$$

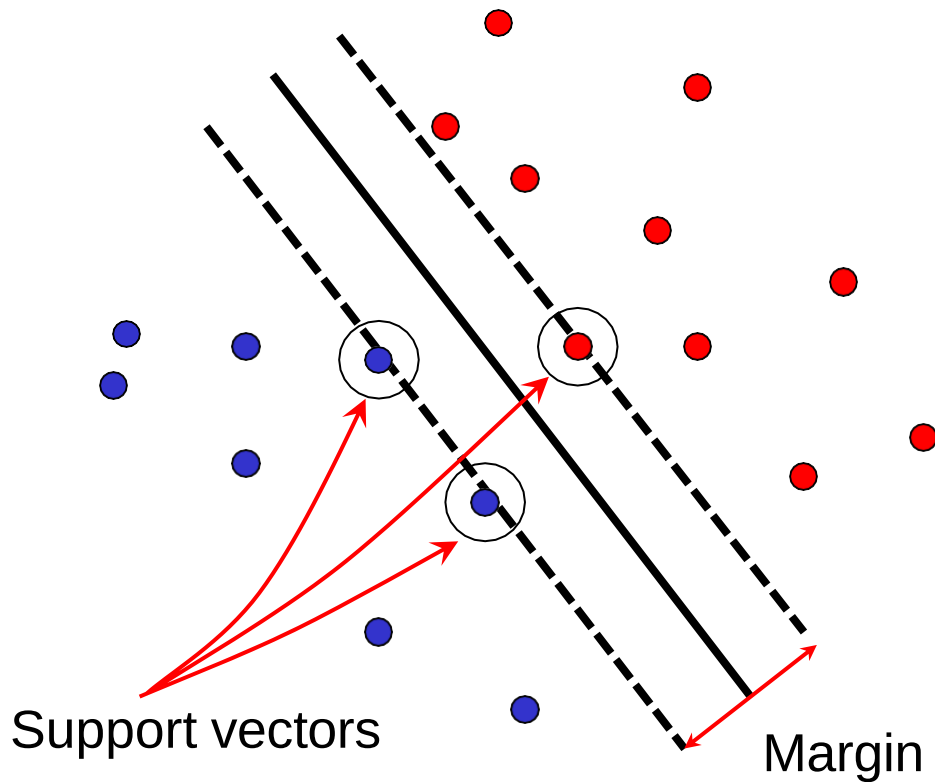
Margins:

$$\mathbf{x} \cdot \mathbf{w} + b = 1$$

$$\mathbf{x} \cdot \mathbf{w} + b = -1$$

Support vector machines

- Find hyperplane that maximizes the *margin* between the positive and negative examples



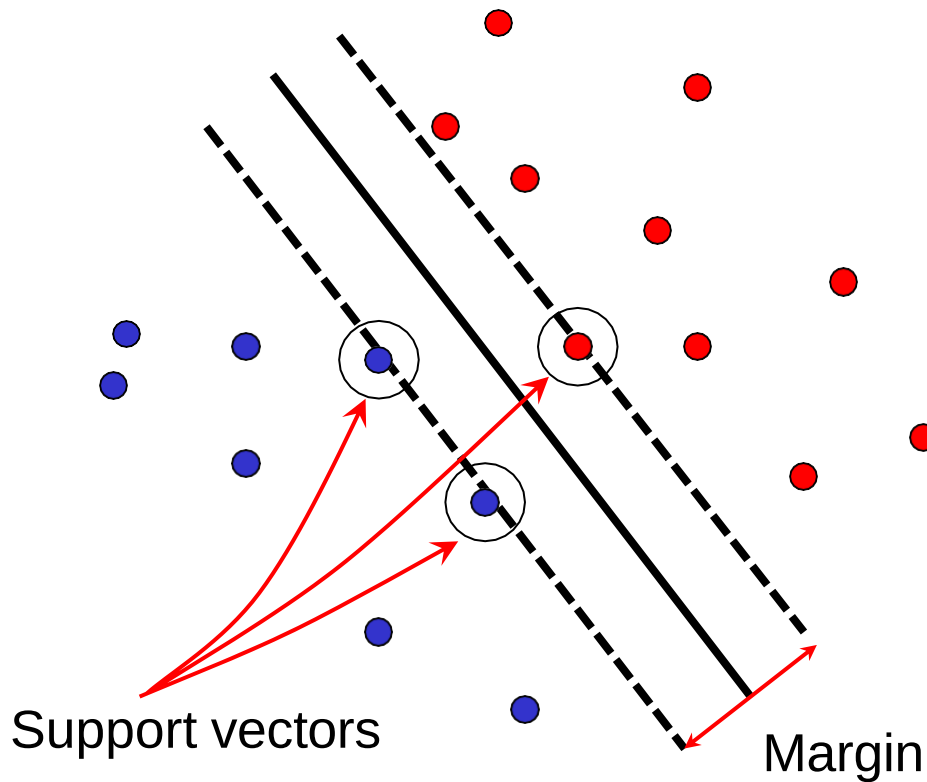
$$\mathbf{x}_i \text{ positive } (y_i = 1): \quad \mathbf{x}_i \cdot \mathbf{w} + b \geq 1$$

$$\mathbf{x}_i \text{ negative } (y_i = -1): \quad \mathbf{x}_i \cdot \mathbf{w} + b \leq -1$$

$$\text{For support vectors,} \quad \mathbf{x}_i \cdot \mathbf{w} + b = \pm 1$$

Support vector machines

- Find hyperplane that maximizes the *margin* between the positive and negative examples



$$\mathbf{x}_i \text{ positive } (y_i = 1): \quad \mathbf{x}_i \cdot \mathbf{w} + b \geq 1$$

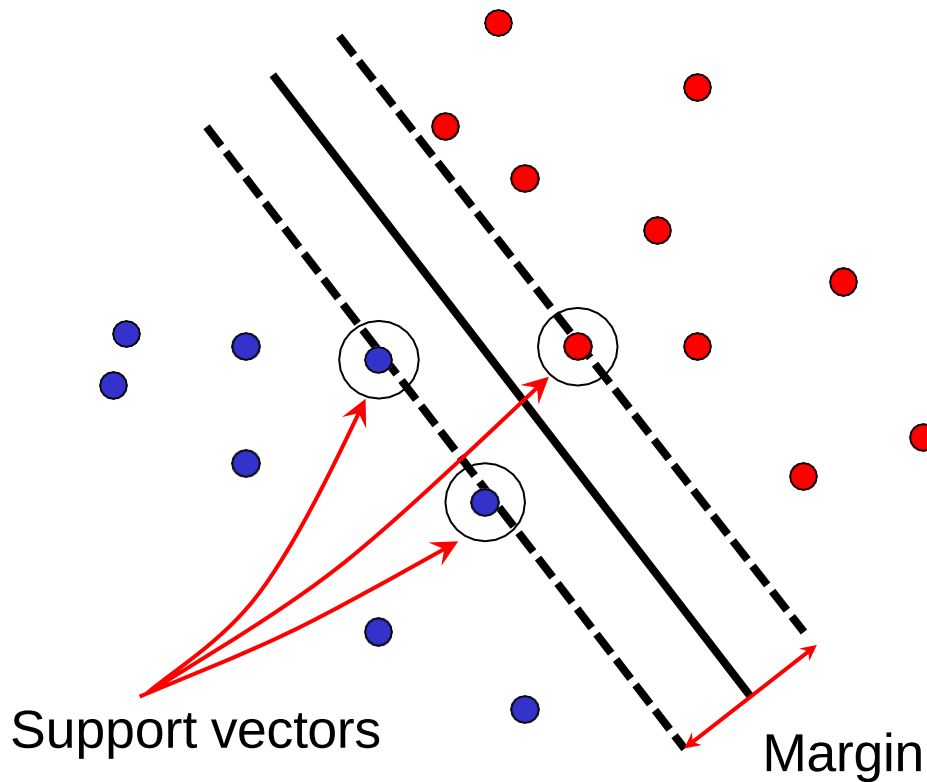
$$\mathbf{x}_i \text{ negative } (y_i = -1): \quad \mathbf{x}_i \cdot \mathbf{w} + b \leq -1$$

$$\text{For support vectors,} \quad \mathbf{x}_i \cdot \mathbf{w} + b = \pm 1$$

$$\text{Distance between point and hyperplane:} \quad \frac{|\mathbf{x}_i \cdot \mathbf{w} + b|}{\|\mathbf{w}\|}$$

Support vector machines

- Find hyperplane that maximizes the *margin* between the positive and negative examples



$$\mathbf{x}_i \text{ positive } (y_i = 1): \quad \mathbf{x}_i \cdot \mathbf{w} + b \geq 1$$

$$\mathbf{x}_i \text{ negative } (y_i = -1): \quad \mathbf{x}_i \cdot \mathbf{w} + b \leq -1$$

$$\text{For support vectors,} \quad \mathbf{x}_i \cdot \mathbf{w} + b = \pm 1$$

$$\text{Distance between point and hyperplane:} \quad \frac{|\mathbf{x}_i \cdot \mathbf{w} + b|}{\|\mathbf{w}\|}$$

$$\text{Therefore, the margin is } 2 / \|\mathbf{w}\|$$

Finding the maximum margin hyperplane

1. Maximize margin $2 / \|\mathbf{w}\|$

Finding the maximum margin hyperplane

1. Maximize margin $2 / \|\mathbf{w}\|$
2. Correctly classify all training data:

$$\mathbf{x}_i \text{ positive } (y_i = 1): \quad \mathbf{x}_i \cdot \mathbf{w} + b \geq 1$$

$$\mathbf{x}_i \text{ negative } (y_i = -1): \quad \mathbf{x}_i \cdot \mathbf{w} + b \leq -1$$

Finding the maximum margin hyperplane

1. Maximize margin $2 / \|\mathbf{w}\|$
2. Correctly classify all training data:

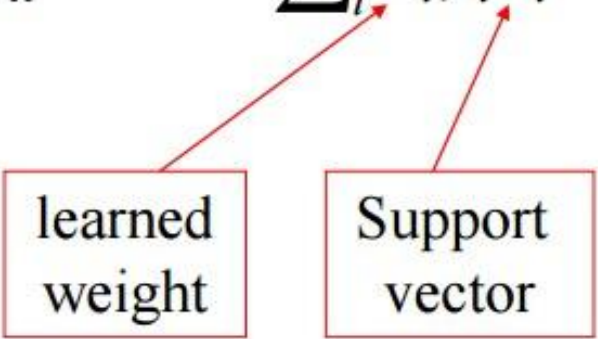
$$\mathbf{x}_i \text{ positive } (y_i = 1): \quad \mathbf{x}_i \cdot \mathbf{w} + b \geq 1$$

$$\mathbf{x}_i \text{ negative } (y_i = -1): \quad \mathbf{x}_i \cdot \mathbf{w} + b \leq -1$$

Quadratic optimization problem:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{subject to} \quad y_i (\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1$$

Finding the maximum margin hyperplane

- Solution: $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$ 

- The weights α_i are non-zero only at support vectors.

Finding the maximum margin hyperplane

- Solution: $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$
 $b = y_i - \mathbf{w} \cdot \mathbf{x}_i$ (for any support vector)
 $\mathbf{w} \cdot \mathbf{x} + b = \sum_i \alpha_i y_i \mathbf{x}_i \cdot \mathbf{x} + b$

Finding the maximum margin hyperplane

- Solution: $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$
 $b = y_i - \mathbf{w} \cdot \mathbf{x}_i$ (for any support vector)

$$\mathbf{w} \cdot \mathbf{x} + b = \sum_i \alpha_i y_i \mathbf{x}_i \cdot \mathbf{x} + b$$

- Classification function:

$$f(\mathbf{x}) = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b) \quad \text{If } f(\mathbf{x}) < 0, \text{ classify as negative,}$$
$$= \text{sign}\left(\sum_i \alpha_i y_i \mathbf{x}_i \cdot \mathbf{x} + b\right) \quad \text{if } f(\mathbf{x}) > 0, \text{ classify as positive}$$



Dot product only!

SVM parameter learning

- Separable data: $\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$ subject to $y_i (\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1$
-
- Maximize margin
- Classify training data correctly

SVM parameter learning

- Separable data: $\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$ subject to $y_i (\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1$

Maximize
margin

Classify training data correctly

- Non-separable data:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \max(0, 1 - y_i (\mathbf{w} \cdot \mathbf{x}_i + b))$$

Maximize
margin

Minimize classification mistakes